Sistema de atención y gestión de turnos para una clínica

Primera entrega: análisis

Curso: Programación IV
Estudiante: Louis Gutierrez-Gandara
Carné: 2501002008
Catedrático: Ingeniero Jorge Tello
Octubre de 2026

Declaración sobre el uso de inteligencia artificial: *Este análisis se elaboró con apoyo de inteligencia artificial (Claude Opus 5.5), utilizada como herramienta para desarrollar el proyecto dentro del plazo establecido. Considero que su uso permite obtener un trabajo de mayor calidad. El proyecto es de mi autoría. Revisé personalmente todo el contenido y respondo por su exactitud y por el cumplimiento de los requisitos planteados por el catedrático.*

Descripción del problema

Una clínica registra a mano los turnos de sus pacientes, y ese método le causa problemas de orden y de control.

Situación actual

Cuando un paciente llega, el personal anota sus datos y su turno sin ninguna herramienta que ordene la espera. Tampoco hay un registro de quién hizo cada anotación ni de los cambios que se hicieron después.

Consecuencias

- Se pierde información de los pacientes y de sus turnos.
- La atención se desordena y no siempre se sabe quién debe pasar después.
- No queda constancia de lo que hizo cada persona del personal, así que un error no se puede rastrear.

Qué necesita la clínica

La clínica necesita una aplicación de escritorio que:

- registre a los pacientes y les genere un turno de forma automática;
- atienda a los pacientes prioritarios antes que a los normales y, dentro de cada grupo, respete el orden de llegada;
- guarde cada acción del personal y permita revertir la última si fue un error;
- conserve la información después de cerrar el programa.

Con esto, la recepción sabe en todo momento quién espera y quién sigue, y el responsable de la clínica puede revisar y corregir lo que hizo el personal.

Alcance

El sistema cubre la recepción de pacientes y el orden en que se atienden. Supone un solo consultorio, así que solo un paciente puede estar en atención a la vez.
Quedan fuera el expediente médico, la facturación, las citas con fecha futura y la impresión de tickets. Son funciones que se podrían agregar en una versión posterior.

Identificación de usuarios

El sistema tiene dos tipos de usuario: Recepcionista y Administrador. Los dos entran con su nombre de usuario y contraseña, y cada uno ve solo las funciones de su rol.

Recepcionista

Es la persona que recibe a los pacientes. Registra a los que llegan por primera vez, genera turnos nuevos para los que regresan y corrige el nombre, la edad o el teléfono de un paciente cuando hace falta. También busca información, revisa las colas de espera y consulta las estadísticas.
En una clínica pequeña la recepción es la que llama a la sala de espera. Por eso el recepcionista también atiende al siguiente paciente y finaliza la atención cuando el consultorio queda libre.

Administrador

Es la persona que supervisa el trabajo de recepción y responde por las correcciones. Puede hacer todo lo que hace el recepcionista y, además, cancelar turnos, deshacer la última acción, revisar el historial de acciones y administrar las cuentas de usuario.

Permisos por rol

El recepcionista puede hacer todo el trabajo diario. Cancelar turnos, deshacer acciones, ver el historial de acciones y administrar usuarios quedan solo para el administrador.

Función	**Administrador**	**Recepcionista**
Iniciar y cerrar sesión	Sí	Sí
Registrar paciente (genera su primer turno)	Sí	Sí
Generar un turno nuevo para un paciente ya registrado	Sí	Sí
Modificar nombre, edad o teléfono de un paciente	Sí	Sí
Buscar información	Sí	Sí
Ver colas y siguiente paciente	Sí	Sí
Atender al siguiente y finalizar la atención	Sí	Sí
Ver historial de pacientes atendidos	Sí	Sí
Ver estadísticas	Sí	Sí

Función	Administrador	Recepcionista
Cancelar un turno	Sí	No
Deshacer la última acción	Sí	No
Ver historial de acciones	Sí	No
Administrar usuarios	Sí	No

Justificación

El recepcionista tiene todo lo que va desde el registro hasta el llamado del paciente, porque en una clínica pequeña la recepción hace ese trabajo de principio a fin. Las funciones que eliminan, revierten o auditan el trabajo de otros quedan con el administrador, para que una sola persona responda por las correcciones.
El historial de acciones registra lo que hizo cada usuario y sirve para supervisar. Por eso tampoco lo ve el recepcionista.
Los permisos se revisan dentro de la lógica del sistema y no solo en la pantalla. Ocultar un botón no basta si otra parte del programa puede pedir la misma función, así que cada operación restringida comprueba el rol del usuario antes de ejecutarse.

Criterios de atención prioritaria

Un turno es prioritario solo si quien registra al paciente elige uno de estos cuatro criterios. Si no elige ninguno, el sistema no acepta la atención prioritaria.

Criterio	Cuándo se usa	Qué revisa el sistema
Adulto mayor	El paciente tiene 60 años o más	Que la edad registrada sea de 60 años o más
Embarazo	La paciente está embarazada	Nada adicional
Discapacidad	El paciente tiene una discapacidad	Nada adicional
Urgencia	Hay signos visibles de gravedad, como dificultad para respirar, sangrado o dolor intenso	Nada adicional

Justificación

Son grupos que no deberían esperar en una fila común, ya sea por su condición física o por el riesgo de que su estado empeore mientras esperan.
El límite de 60 años sigue la Ley de Protección para las Personas de la Tercera Edad de Guatemala (Decreto 80-96), cuyo artículo 3 considera de la tercera edad a toda persona de 60 años o más. Es el único criterio que el sistema puede comprobar con un dato que ya tiene, la edad del paciente.
Pedir un criterio obliga a quien registra a justificar la prioridad. Así se evita que todos los

pacientes terminen en la cola prioritaria y que la cola normal no avance nunca.

Requerimientos

Cómo se redactaron

Un requerimiento es una necesidad, función, condición o característica que el sistema debe cumplir. Cada uno está escrito de forma clara y específica, relacionado con el problema de la clínica, y de modo que una persona pueda comprobar si se cumplió.

Grupo	Responde a	Forma de redacción
Requerimientos funcionales (RF)	Qué hace el sistema	"El sistema deberá" + acción + información o condición
Requerimientos no funcionales (RNF)	Cómo debe hacerlo: usabilidad, rendimiento, seguridad, confiabilidad, compatibilidad, persistencia y mantenibilidad	Una condición que se puede medir o comprobar
Reglas del negocio (RN)	Qué normas propias de la clínica debe respetar	Una norma de la organización

Para clasificar una frase se hacen tres preguntas. Si describe una acción del sistema, es funcional. Si describe calidad, rendimiento, seguridad o tecnología, es no funcional. Si describe una norma de la clínica, es regla del negocio.
Cada requerimiento se revisó con cinco preguntas: ¿es claro, específico, verificable, necesario y posible? Las funciones generales, como "administrar pacientes", se dividieron en funciones concretas. Los requerimientos describen funciones y no elementos de pantalla; los botones, tablas y menús se definen en los bocetos de la interfaz.

Requerimientos funcionales

El sistema tiene 44 requerimientos funcionales, agrupados en nueve módulos.

Inicio de sesión y usuarios

ID	Requerimiento
RF-01	El sistema deberá permitir iniciar sesión con nombre de usuario y contraseña.
RF-02	El sistema deberá verificar las credenciales antes de dar acceso y mostrar un mensaje si son incorrectas.
RF-03	El sistema deberá mostrar solo las funciones permitidas para el rol del usuario que inició sesión.

ID	Requerimiento
RF-04	El sistema deberá permitir cerrar sesión y volver a la ventana de inicio de sesión.
RF-05	El sistema deberá permitir salir de la aplicación después de pedir confirmación.
RF-06	El sistema deberá permitir al administrador crear usuarios con nombre completo, nombre de usuario, contraseña y rol.
RF-07	El sistema deberá permitir al administrador eliminar usuarios, excepto su propia cuenta.
RF-08	El sistema deberá mostrar al administrador la lista de usuarios con su nombre y rol.

Pacientes

ID	Requerimiento
RF-09	El sistema deberá permitir registrar a un paciente con su código, nombre completo, edad, teléfono, motivo de la consulta y tipo de atención.
RF-10	El sistema deberá asignar automáticamente la fecha y hora de registro del paciente.
RF-11	El sistema deberá proponer el siguiente código de paciente disponible.
RF-12	El sistema deberá rechazar el registro si el código del paciente ya existe.
RF-13	El sistema deberá pedir un criterio de prioridad cuando el tipo de atención sea Prioritaria.
RF-14	El sistema deberá permitir modificar el nombre, la edad y el teléfono de un paciente registrado, sin cambiar su código.
RF-15	El sistema deberá mostrar la lista de pacientes registrados.

Turnos

ID	Requerimiento
RF-16	El sistema deberá generar automáticamente

ID	Requerimiento
	un turno al registrar a un paciente, con número único, datos del paciente, tipo de atención, motivo, fecha y hora de generación y estado Pendiente.
RF-17	El sistema deberá permitir generar un turno nuevo para un paciente ya registrado, indicando motivo y tipo de atención.
RF-18	El sistema deberá permitir consultar el estado de un turno a partir de su número.
RF-19	El sistema deberá permitir cancelar un turno en estado Pendiente, después de pedir confirmación.

Colas

ID	Requerimiento
RF-20	El sistema deberá agregar cada turno nuevo al final de la cola que corresponde a su tipo de atención.
RF-21	El sistema deberá mostrar los turnos pendientes de cada cola en orden de llegada.
RF-22	El sistema deberá mostrar cuál es el siguiente turno que será atendido.
RF-23	El sistema deberá atender al siguiente turno tomándolo de la cola prioritaria, o de la cola normal si la prioritaria está vacía.
RF-24	El sistema deberá permitir finalizar la atención del turno en curso, registrando la fecha y hora de atención y el usuario.
RF-25	El sistema deberá mostrar la cantidad de turnos en cada cola.
RF-26	El sistema deberá informar cuando las dos colas estén vacías.

Historial de acciones

ID	Requerimiento
RF-27	El sistema deberá registrar cada acción con su tipo, descripción, usuario responsable,

ID	Requerimiento
	información relacionada y fecha y hora.
RF-28	El sistema deberá mostrar la última acción registrada.
RF-29	El sistema deberá mostrar el historial de acciones empezando por la más reciente.
RF-30	El sistema deberá deshacer la última acción deshacible y devolver los datos a su estado anterior.
RF-31	El sistema deberá informar cuando no haya acciones registradas o ninguna se pueda deshacer.

Búsquedas

ID	Requerimiento
RF-32	El sistema deberá buscar turnos por código de paciente, nombre del paciente, número de turno, estado y tipo de atención.
RF-33	El sistema deberá aceptar coincidencias parciales en la búsqueda por nombre, sin distinguir mayúsculas de minúsculas ni tildes.
RF-34	El sistema deberá mostrar un mensaje cuando la búsqueda no tenga resultados.

Historial de atendidos y estadísticas

ID	Requerimiento
RF-35	El sistema deberá conservar un registro de cada atención finalizada con turno, paciente, tipo de atención, fecha y hora de registro, fecha y hora de atención y usuario que atendió.
RF-36	El sistema deberá mostrar el historial de atendidos ordenado por fecha de atención, del más reciente al más antiguo.
RF-37	El sistema deberá mostrar el total de pacientes registrados y la cantidad de turnos pendientes, en atención, atendidos, cancelados, normales y prioritarios.

ID	Requerimiento
RF-38	El sistema deberá mostrar la cantidad de turnos en cada cola dentro de las estadísticas.
RF-39	El sistema deberá actualizar las estadísticas después de cada operación que cambie los datos.

Almacenamiento

ID	Requerimiento
RF-40	El sistema deberá guardar usuarios, pacientes, turnos e historial de atendidos en archivos JSON después de cada operación que los modifique.
RF-41	El sistema deberá cargar la información guardada al iniciar y reconstruir las colas con los turnos pendientes.
RF-42	El sistema deberá crear los archivos que no existan y los usuarios iniciales cuando falte el archivo de usuarios.

Validación de datos

ID	Requerimiento
RF-43	El sistema deberá rechazar los datos que no cumplan su formato (código, nombre, edad, teléfono, motivo, nombre de usuario y contraseña) e indicar qué campo debe corregirse.
RF-44	El sistema deberá revisar al cargar los turnos que ningún número esté repetido y avisar al usuario si encuentra uno.

Requerimientos no funcionales

Los 22 requerimientos no funcionales fijan cómo debe comportarse el sistema en siete categorías.

ID	Categoría	Requerimiento
RNF-01	Usabilidad	Los campos, botones y columnas deberán usar los términos de la clínica

ID	Categoría	Requerimiento
		(paciente, turno, cola, atención) y ningún término técnico de programación.
RNF-02	Usabilidad	La información de pacientes, turnos y acciones deberá presentarse en tablas con encabezados.
RNF-03	Usabilidad	Cualquier función deberá estar a un máximo de dos clics desde la ventana principal.
RNF-04	Usabilidad	Cancelar un turno, eliminar un usuario, deshacer una acción y salir deberán pedir confirmación.
RNF-05	Rendimiento	Las búsquedas y la actualización de tablas deberán responder en menos de dos segundos con 1000 turnos registrados.
RNF-06	Seguridad	La contraseña deberá mostrarse oculta mientras se escribe.
RNF-07	Seguridad	Las contraseñas deberán guardarse como hash con sal y nunca en texto plano.
RNF-08	Seguridad	Las funciones restringidas deberán bloquearse en la lógica del sistema, además de ocultarse en la interfaz.
RNF-09	Seguridad	El mensaje de credenciales incorrectas no deberá indicar si falló el usuario o la contraseña.
RNF-10	Confiabilidad	Una entrada incorrecta no deberá cerrar la aplicación.
RNF-11	Confiabilidad	Los errores deberán mostrarse con mensajes comprensibles, sin detalles

ID	Categoría	Requerimiento
		técnicos. Los detalles se escribirán en errors.log.
RNF-12	Confiabilidad	Si un archivo JSON está dañado, el sistema deberá guardar una copia del archivo, avisar al usuario y seguir funcionando.
RNF-13	Confiabilidad	El guardado deberá hacerse con un archivo temporal para que una falla no deje un archivo a medio escribir.
RNF-14	Compatibilidad	La aplicación deberá desarrollarse en Python 3.11 o superior con Tkinter, sin librerías externas.
RNF-15	Compatibilidad	La aplicación deberá funcionar en Windows 10 y 11.
RNF-16	Persistencia	La información deberá conservarse después de cerrar el programa.
RNF-17	Persistencia	Los archivos JSON deberán leerse y escribirse en UTF-8, nunca en ASCII ni en la codificación del sistema, con las tildes y la ñ guardadas como caracteres reales y con sangría, para que una persona pueda leerlos.
RNF-18	Mantenibilidad	La lógica deberá estar separada de la interfaz, en paquetes distintos.
RNF-19	Mantenibilidad	Los identificadores del código deberán escribirse en inglés, con clases en PascalCase y métodos y variables en snake_case con nombres descriptivos.
RNF-20	Usabilidad	Todos los textos que ve el usuario (etiquetas, botones,

ID	Categoría	Requerimiento
		columnas, menús, mensajes y valores como estados o tipos de atención) deberán estar en español.
RNF-21	Confiabilidad	Si falla el guardado de un archivo, el sistema deberá avisar al usuario y conservar en memoria los datos de la sesión.
RNF-22	Persistencia	El historial de acciones deberá conservarse mientras la aplicación esté abierta, aunque cambie el usuario, y no se guardará en archivo.

El historial de acciones queda solo en memoria porque cada acción deshacible apunta a objetos que existen mientras el programa corre. Al cerrar, los archivos JSON pasan a ser el estado definitivo y ya no queda nada por revertir.

Reglas del negocio

Estas 12 reglas son normas de la clínica. El sistema las aplica en cada operación, sin importar desde qué pantalla se llame.

ID	Regla
RN-01	Los pacientes prioritarios se atienden antes que los normales.
RN-02	Los pacientes de una misma cola se atienden en orden de llegada.
RN-03	El código del paciente no puede repetirse.
RN-04	El número de turno no puede repetirse ni volver a usarse.
RN-05	Solo se puede cancelar un turno Pendiente.
RN-06	Un turno Atendido o Cancelado no cambia de estado, salvo al deshacer la acción que lo llevó ahí.
RN-07	Un paciente no puede tener más de un turno Pendiente o En atención.
RN-08	Solo un turno puede estar En atención a la vez.

ID	Regla
RN-09	La atención prioritaria requiere un criterio: Adulto mayor, Embarazo, Discapacidad o Urgencia. El criterio Adulto mayor requiere 60 años o más.
RN-10	Solo el administrador puede cancelar turnos, deshacer acciones, ver el historial de acciones y administrar usuarios.
RN-11	El inicio de sesión, el cierre de sesión, la administración de usuarios y el propio deshacer quedan solo como evidencia y no se pueden deshacer.
RN-12	Siempre debe existir al menos un administrador, y nadie puede eliminar su propia cuenta.

Función, regla y condición

Una función casi nunca va sola: la acompañan reglas de la clínica y condiciones de calidad. Tres ejemplos del sistema:

Requerimiento funcional	Regla del negocio	Requerimiento no funcional
RF-23 Atender al siguiente turno	RN-01 y RN-02 Prioritarios primero, en orden de llegada	RNF-10 Una cola vacía no cierra el programa
RF-19 Cancelar un turno	RN-05 y RN-10 Solo turnos Pendiente y solo el administrador	RNF-04 Se pide confirmación
RF-09 Registrar paciente	RN-03 Código único	RNF-16 Los datos se conservan al cerrar

Cobertura del enunciado

Los doce puntos que pide el enunciado quedan cubiertos por al menos un requerimiento.

Punto del enunciado	Qué pide	Requerimientos que lo cubren
1. Inicio de sesión	Credenciales, dos tipos de usuario con permisos distintos	RF-01 a RF-08, RNF-06, RNF-07, RNF-09, RN-10, RN-12
2. Registro de pacientes	Datos del paciente, código único, criterios de prioridad	RF-09 a RF-15, RF-43, RN-03, RN-09
3. Generación de turnos	Turno automático, número único, cuatro estados	RF-16 a RF-19, RF-24, RN-04 a RN-07

Punto del enunciado	Qué pide	Requerimientos que lo cubren
4. Administración de colas	Dos colas FIFO, prioritarios primero y las ocho operaciones pedidas	RF-18 a RF-26, RN-01, RN-02, RN-08
5. Historial con pila	Agregar, consultar la última, mostrar, deshacer, avisar si no hay acciones	RF-27 a RF-31, RN-06, RN-11, RNF-22
6. Búsquedas	Al menos tres criterios y mensaje sin resultados	RF-32 a RF-34
7. Historial de atendidos	Registro ordenado de cada atención	RF-35, RF-36
8. Estadísticas	Conteos de pacientes, turnos y colas, actualizados	RF-37 a RF-39
9. Interfaz gráfica	Ventanas, menús, tablas, mensajes de confirmación y error	RF-03 a RF-05, RNF-01 a RNF-04, RNF-14, RNF-20
10. Validaciones	Datos obligatorios y con formato, duplicados, permisos, confirmaciones	RF-02, RF-12, RF-13, RF-26, RF-31, RF-42 a RF-44, RN-03, RN-04, RN-07 a RN-10, RNF-04, RNF-08, RNF-10
11. Manejo de excepciones	Datos numéricos, archivos inexistentes o dañados, errores al guardar, estructuras vacías, errores inesperados	RF-26, RF-31, RF-42, RNF-10 a RNF-13, RNF-21
12. Almacenamiento en JSON	Guardar y cargar usuarios, pacientes, turnos y atendidos; decidir qué pasa con la pila	RF-40 a RF-42, RF-44, RNF-16, RNF-17, RNF-22

Dos partes del enunciado se detallan en otras secciones de esta entrega. Los elementos concretos de la pantalla (botones, Treeview, Combobox) están en los bocetos de la interfaz, y la validación de seleccionar un registro antes de modificarlo o cancelarlo está en la sección de validaciones.

Análisis de pilas y colas

Estructuras que usa el sistema

El sistema usa dos colas para ordenar los turnos y dos pilas para el historial de acciones. Las colas deciden quién pasa al consultorio: una guarda los turnos prioritarios y la otra los normales. Las pilas guardan lo que hace el personal y permiten revertir la última acción deshacible.
Las colas se implementan en la clase TicketQueue sobre collections.deque, y las pilas en la clase Stack sobre una lista de Python. Atender, cancelar y deshacer pasan por estas estructuras, así que forman parte del funcionamiento real del programa.

Por qué los turnos van en una cola

En la sala de espera, quien llega primero pasa primero. Ese es el principio FIFO (primero en entrar, primero en salir), y una cola lo aplica sin pasos extra: cada turno nuevo entra al final y el siguiente en ser atendido siempre está al frente. Con una lista común habría que buscar, en cada atención, cuál turno tiene la fecha más antigua.
La cola también responde las preguntas que la recepción se hace durante el día: quién sigue, cuántos esperan y si queda alguien. Cada una corresponde a una operación de TicketQueue.

Por qué dos colas y no una lista ordenada

El sistema tiene una cola para la atención prioritaria y otra para la normal. Los prioritarios pasan antes que los normales (RN-01), y dentro de cada grupo se respeta el orden de llegada (RN-02).
Con una sola lista habría que reordenarla cada vez que llega un prioritario: buscar dónde termina el bloque de prioritarios e insertar ahí el turno nuevo. Con dos colas, cada turno entra al final de la suya y ningún otro turno cambia de lugar. Elegir al siguiente se reduce a una pregunta: si la cola prioritaria tiene turnos, sale el de su frente; si está vacía, sale el del frente de la normal.

Operación	**Una lista ordenada**	**Dos colas sobre deque**
Agregar un turno	O(n): buscar la posición e insertar	O(1): agregar al final de su cola
Sacar al siguiente	O(n): pop(0) mueve todos los elementos	O(1): popleft en la cola que corresponde
Contar por tipo	O(n): recorrer y filtrar	O(1): longitud de cada cola

O(1) significa que la operación tarda lo mismo con 3 turnos que con 1000. O(n) significa que el trabajo crece con la cantidad de turnos, porque hay que recorrerlos. Se usa collections.deque, que el enunciado permite, porque agrega y saca por ambos extremos en tiempo constante. Una lista de Python saca del inicio en O(n), ya que tiene que mover los demás elementos una posición.
TicketManager decide la cola de cada turno en su método _queue_for. Ese método consulta

care_type.is_priority sin preguntar de qué clase es el objeto, así que un tipo de atención nuevo funcionaría sin cambiar el método.

Operaciones de TicketQueue

TicketQueue guarda los turnos en un deque protegido (_items) y solo los expone mediante sus métodos. Ninguna otra clase toca el deque directamente.

Método	**Qué hace**	**Si la cola está vacía**	**Costo**	**Dónde se usa**
enqueue(ticket)	Agrega el turno al final	El turno queda como único elemento	O(1)	Registrar paciente y generar turno
dequeue()	Saca y devuelve el turno del frente	Lanza EmptyQueueError	O(1)	Atender siguiente
peek()	Devuelve el turno del frente sin sacarlo	Lanza EmptyQueueError	O(1)	Mostrar quién sigue
is_empty()	Indica si la cola no tiene turnos	Devuelve True	O(1)	Elegir de qué cola sale el siguiente
count()	Devuelve cuántos turnos hay	Devuelve 0	O(1)	Contadores y estadísticas
list_all()	Devuelve una copia de los turnos en orden de llegada	Devuelve una lista vacía	O(n)	Tablas de pendientes
remove(ticket)	Saca un turno de cualquier posición y devuelve la posición que tenía	Lanza NotFoundError	O(n)	Cancelar turno y deshacer un registro
insert_at(position, ticket)	Inserta el turno en una posición dada	El turno queda como único elemento	O(n)	Deshacer una cancelación
push_front(ticket)	Coloca el turno al frente	El turno queda como único elemento	O(1)	Deshacer una atención

Las operaciones del punto 4 del enunciado quedan cubiertas así:

Operación del enunciado	Cómo se resuelve
Agregar un paciente a su cola	enqueue, llamado desde TicketManager.create_ticket
Mostrar los pendientes	list_all de cada cola
Consultar quién sigue	TicketManager.peek_next, que usa is_empty y peek
Atender al siguiente	TicketManager.call_next, que usa dequeue
Mostrar la cantidad en cada cola	count de cada cola
Consultar el estado de un turno	TicketManager.get_status, que lo busca por número en el diccionario de turnos
Cancelar un turno pendiente	TicketManager.cancel_ticket, que usa remove
Indicar que las colas están vacías	TicketManager.queues_empty, que usa is_empty en las dos

Una cola vacía nunca cierra el programa. peek_next revisa is_empty antes de mirar el frente y devuelve None si no hay nadie. Si se intenta atender con las dos colas vacías, call_next deja pasar la EmptyQueueError y la interfaz muestra "No hay pacientes en espera."

Cancelar un turno que está en medio de la cola

Una cola pura solo deja sacar elementos del frente, pero un paciente puede irse de la sala aunque no sea el siguiente. Para ese caso TicketQueue tiene remove(ticket): recorre la cola hasta encontrar el turno, lo saca y devuelve la posición que tenía, contando desde 0 en el frente. El recorrido cuesta O(n). Es aceptable porque las cancelaciones son pocas y la cola de una clínica de un consultorio tiene pocas decenas de turnos.
TicketManager.cancel_ticket(number) revisa primero que el turno esté Pendiente (RN-05). Luego lo saca con remove, lo pasa a Cancelado y devuelve la posición. ClinicSystem guarda esa posición dentro de la acción CancelTicketAction.
Si el administrador deshace la cancelación, restore_cancelled llama a insert_at(posición, turno) y el turno vuelve a su lugar. La posición guardada sigue siendo válida porque la pila obliga a deshacer en orden inverso: cualquier cambio a las colas hecho después de la cancelación se revierte antes que ella, así que la cola está igual que justo después de cancelar.

Numeración de turnos y reconstrucción de las colas

Cada turno lleva un prefijo según su tipo (N para normal, P para prioritario) y un número tomado de un contador global de tres dígitos o más: N-001, N-002, P-003, N-004. Como el contador es uno solo para las dos colas, ningún número se repite entre ellas (RN-04). Si se deshace el registro de un turno, su número no se vuelve a usar, para no confundir a un paciente que ya lo escuchó o lo anotó.
Las colas no se guardan como tales en los archivos JSON. Al iniciar, TicketManager.load lee tickets.json, toma los turnos en estado Pendiente, los ordena por la parte numérica del número

(la propiedad sequence) y encola cada uno en la cola de su tipo. El contador crece con cada turno generado, así que ordenar por número equivale a ordenar por llegada y las colas quedan como estaban antes de cerrar.
El contador se recalcula como el mayor número existente más uno, sin guardarlo aparte. El turno que estaba En atención se recupera por separado y no entra a ninguna cola. Las reversiones también respetan el orden por número: un turno que vuelve al frente o a su posición anterior queda en el mismo lugar que tendría si se ordenara de nuevo al iniciar.

Por qué el historial usa una pila

Deshacer siempre revierte lo último que se hizo. Si el administrador cancela un turno y luego atiende al siguiente, el primer deshacer revierte la atención y el segundo la cancelación. Ese orden es LIFO (último en entrar, primero en salir), y en una pila la acción más reciente siempre está en la cima.
La clase Stack guarda los elementos en una lista de Python protegida (_items), con la cima al final. Agregar y quitar por el final de una lista cuesta O(1), así que aquí no hace falta deque.

Método	**Qué hace**	**Si la pila está vacía**	**Costo**
push(item)	Coloca un elemento en la cima	El elemento queda como único	O(1)
pop()	Quita y devuelve el elemento de la cima	Lanza EmptyStackError	O(1)
peek()	Devuelve la cima sin quitarla	Lanza EmptyStackError	O(1)
is_empty()	Indica si no hay elementos	Devuelve True	O(1)
count()	Devuelve cuántos elementos hay	Devuelve 0	O(1)
list_from_top()	Devuelve una copia desde la cima hasta el fondo	Devuelve una lista vacía	O(n)

Las operaciones del punto 5 del enunciado quedan cubiertas así:

Operación del enunciado	**Cómo se resuelve**
Agregar una acción	ActionHistory.register, que usa push
Consultar la última acción	ActionHistory.last_action, que usa peek sobre la pila de historial
Mostrar el historial desde la más reciente	ActionHistory.list_all, que usa list_from_top
Deshacer la última acción	ActionHistory.undo_last, que usa pop sobre la pila de deshacer
Indicar que no hay acciones	has_actions y can_undo, que usan is_empty

Si la pila de deshacer está vacía, pop lanza EmptyStackError y la interfaz muestra "No hay acciones que se puedan deshacer." Si la reversión falla a medias, undo_last vuelve a apilar la acción y relanza el error, de modo que se puede intentar otra vez.

Qué guarda cada acción y cuáles se pueden deshacer

Cada elemento de la pila es un objeto Action con cinco datos: tipo de acción, descripción, usuario responsable, información relacionada y fecha y hora en formato AAAA-MM-DD HH:MM:SS. Las acciones deshacibles guardan además lo que necesitan para revertirse.

Acción	**¿Se deshace?**	**Qué guarda para revertirse**	**Cómo se revierte**
Iniciar sesión	No	Nada adicional	Queda como evidencia
Cerrar sesión	No	Nada adicional	Queda como evidencia
Registrar paciente	Sí	El paciente y su primer turno	Se eliminan el paciente y el turno, y el turno sale de su cola
Generar turno para un paciente existente	Sí	El turno	El turno se elimina y sale de su cola
Modificar paciente	Sí	El paciente y su nombre, edad y teléfono anteriores	Se restauran los valores anteriores
Atender siguiente	Sí	El turno	Vuelve a Pendiente y se coloca al frente de su cola
Finalizar atención	Sí	El turno y su registro de atención	Vuelve a En atención y el registro sale del historial de atendidos
Cancelar turno	Sí	El turno y la posición que tenía en la cola	Vuelve a Pendiente en la misma posición
Crear o eliminar usuario	No	Nada adicional	Se corrige desde la administración de usuarios
Deshacer	No	La acción que se revirtió	Queda como evidencia con el tipo Acción deshecha

Se pueden deshacer las acciones que cambian pacientes, turnos o colas, porque un error en ellas altera el orden de atención y cada una tiene una forma clara de revertirse. El inicio y el

cierre de sesión ya ocurrieron y no dejan ningún dato que devolver. La administración de usuarios tiene su propia pantalla para crear y eliminar cuentas, así que sus errores se corrigen ahí. El deshacer tampoco se deshace, porque el sistema no tiene opción de rehacer.
Cada clase hija de UndoableAction implementa su propio _revert, y ActionHistory solo llama a action.undo(context) sin saber qué tipo de acción tiene en la mano. Después de revertir, el sistema vuelve a guardar los archivos JSON.

Dos pilas dentro del historial

ActionHistory contiene dos objetos de la misma clase Stack. La pila de historial guarda todas las acciones y sirve para mostrarlas y consultar la última. La pila de deshacer guarda solo las acciones deshacibles.
Con una sola pila, las acciones de evidencia taparían a las deshacibles. Si el administrador cancela un turno por error, cierra sesión y vuelve a entrar, la cima de una pila única sería el inicio de sesión. Esa acción no se puede deshacer, y la cancelación quedaría bloqueada debajo de ella. Con dos pilas, el inicio de sesión va solo a la pila de historial y la cancelación sigue en la cima de la pila de deshacer. El recorrido 2 muestra este caso paso a paso.
ActionHistory.register siempre apila en la pila de historial y, si la acción es deshacible, también en la de deshacer. Al deshacer, la acción sale de la pila de deshacer, queda marcada como deshecha en la de historial y se apila una acción nueva de evidencia con el tipo Acción deshecha. Las dos pilas siguen el orden LIFO.

La pila vive solo durante la ejecución

Las dos pilas existen mientras la aplicación está abierta. Se conservan si un usuario cierra sesión y entra otro, pero no se guardan en ningún archivo.
Cada acción deshacible guarda referencias a objetos en memoria, como el turno o el paciente. Guardarlas en JSON obligaría a reconstruir esas referencias al iniciar y a comprobar si cada acción todavía tiene sentido con los datos cargados. Al cerrar el programa, los archivos JSON pasan a ser el estado definitivo y no queda nada pendiente de revertir.
La limitación es que el historial de acciones se pierde al cerrar. Si se quisiera conservarlo como bitácora, podría escribirse en un archivo aparte, solo para consulta y sin opción de deshacer. Queda como mejora posible.

Recorrido 1: llegada de cuatro turnos y tres atenciones

Llegan cuatro pacientes en este orden: tres de atención normal y uno prioritario en tercer lugar. Se generan N-001, N-002, P-003 y N-004. Luego se atiende tres veces, y cada atención se finaliza antes de llamar al siguiente porque solo hay un consultorio (RN-08). El orden esperado es P-003, N-001 y N-002.
El administrador ya inició sesión, así que la pila de historial empieza con Ini (inicio de sesión) en el fondo. Las colas se leen del frente al final y las pilas de la cima al fondo. Abreviaturas: Reg es registro de paciente, Atn es atender siguiente y Fin es finalizar atención.

Paso	Operación	Cola prioritaria	Cola normal	En atención	Pila de deshacer
0	Estado inicial	vacía	vacía	ninguno	vacía
1	Registrar PAC-0001, Normal. Se genera N-001	vacía	N-001	ninguno	Reg N-001
2	Registrar PAC-0002, Normal. Se genera N-002	vacía	N-001, N-002	ninguno	Reg N-002, Reg N-001
3	Registrar PAC-0003, Prioritaria. Se genera P-003	P-003	N-001, N-002	ninguno	Reg P-003, Reg N-002, Reg N-001
4	Registrar PAC-0004, Normal. Se genera N-004	P-003	N-001, N-002, N-004	ninguno	Reg N-004, Reg P-003, Reg N-002, Reg N-001
5	Atender siguiente. La prioritaria no está vacía: dequeue de P-003	vacía	N-001, N-002, N-004	P-003	Atn P-003, Reg N-004, Reg P-003, Reg N-002, Reg N-001
6	Finalizar atención. P-003 pasa a Atendido	vacía	N-001, N-002, N-004	ninguno	Fin P-003, Atn P-003, Reg N-004, Reg P-003, Reg N-002, Reg N-001
7	Atender siguiente. La prioritaria está vacía: dequeue de N-001	vacía	N-002, N-004	N-001	Atn N-001, Fin P-003, Atn P-003, Reg N-004, Reg P-003, Reg N-002, Reg N-001
8	Finalizar atención. N-001 pasa a	vacía	N-002, N-004	ninguno	Fin N-001, Atn N-001, Fin P-003, Atn

Paso	Operación	Cola prioritaria	Cola normal	En atención	Pila de deshacer
	Atendido				P-003, Reg N-004, Reg P-003, Reg N-002, Reg N-001
9	Atender siguiente. La prioritaria está vacía: dequeue de N-002	vacía	N-004	N-002	Atn N-002, Fin N-001, Atn N-001, Fin P-003, Atn P-003, Reg N-004, Reg P-003, Reg N-002, Reg N-001
10	Finalizar atención. N-002 pasa a Atendido	vacía	N-004	ninguno	Fin N-002, Atn N-002, Fin N-001, Atn N-001, Fin P-003, Atn P-003, Reg N-004, Reg P-003, Reg N-002, Reg N-001

P-003 pasó primero aunque llegó tercero, porque en el paso 5 la cola prioritaria tenía un turno. En los pasos 7 y 9 esa cola ya estaba vacía, y los normales salieron en su orden de llegada. N-004 queda pendiente al frente de la cola normal.

Todas las acciones del recorrido son deshacibles, así que la pila de historial tiene los mismos diez elementos que la de deshacer, más Ini en el fondo. Los pasos 6, 8 y 10 agregan además un registro al historial de atendidos, con la fecha y hora de atención y el usuario.

Recorrido 2: cancelar N-002 y deshacer

El recorrido parte del estado del paso 4 del recorrido 1, antes de atender a nadie. La cola normal tiene N-001 en la posición 0, N-002 en la 1 y N-004 en la 2. El administrador cancela N-002, cierra sesión, vuelve a entrar y deshace. La cola prioritaria conserva P-003 todo el tiempo y no hay nadie en atención.

Además de las abreviaturas del recorrido 1: Can es cancelación de turno, Cie es cierre de sesión y Desh es acción deshecha. En la cola normal, cada turno lleva su posición delante.

Paso	Operación	Cola normal	Estado de N-002	Pila de deshacer	Pila de historial
0	Estado inicial	0: N-001, 1: N-002, 2: N-004	Pendiente	Reg N-004, Reg P-003, Reg N-002, Reg N-001	Reg N-004, Reg P-003, Reg N-002, Reg N-001, Ini
1	Cancelar N-002 y confirmar. remove devuelve la posición 1, que se guarda en la acción	0: N-001, 1: N-004	Cancelado	Can N-002 (posición 1), Reg N-004, Reg P-003, Reg N-002, Reg N-001	Can N-002, Reg N-004, Reg P-003, Reg N-002, Reg N-001, Ini
2	Cerrar sesión	0: N-001, 1: N-004	Cancelado	Sin cambios	Cie, Can N-002, Reg N-004, Reg P-003, Reg N-002, Reg N-001, Ini
3	Iniciar sesión otra vez como administrador	0: N-001, 1: N-004	Cancelado	Sin cambios	Ini, Cie, Can N-002, Reg N-004, Reg P-003, Reg N-002, Reg N-001, Ini
4	Deshacer y confirmar. pop saca Can N-002 y restore_cancelled llama a insert_at(1, N-002)	0: N-001, 1: N-002, 2: N-004	Pendiente	Reg N-004, Reg P-003, Reg N-002, Reg N-001	Desh, Ini, Cie, Can N-002 (deshecha), Reg N-004, Reg P-003, Reg N-002, Reg N-001, Ini

En el paso 4, la cima de la pila de historial es un inicio de sesión, pero el deshacer trabaja sobre la otra pila, cuya cima sigue siendo la cancelación. N-002 vuelve a la posición 1, entre N-001 y N-004, y la cola queda igual que en el paso 0.

La pila de deshacer termina como empezó. La de historial conserva la cancelación marcada como deshecha y suma la acción Desh, así el administrador puede ver qué pasó y quién lo hizo. Después de deshacer, tickets.json se guarda de nuevo con N-002 en estado Pendiente; si se reinicia el programa, el orden por número deja la cola normal otra vez como N-001, N-002, N-004.

Propuesta inicial de clases

El sistema se reparte en seis paquetes: modelos, estructuras de datos, persistencia, servicios, utilidades e interfaz. Cada tabla indica qué hace cada clase, qué datos guarda y qué métodos ofrece. Las firmas completas, con parámetros y tipos, se fijan al implementar cada clase, y el diagrama UML con sus relaciones forma parte de la segunda entrega.
Los nombres de clases, atributos y métodos están en inglés, como el resto del código. Lo que ve el personal de la clínica (estados, tipos de atención, mensajes y botones) está en español.
Un guion bajo al inicio marca un atributo o método protegido (_items) y dos marcan uno privado (__password_hash). Nada de eso se modifica desde fuera de su clase sin pasar por una propiedad o un método.

Personas, usuarios y pacientes

Administradores, recepcionistas y pacientes heredan de Person, una clase abstracta que guarda el nombre completo. Los dos roles del personal se distinguen por los permisos que devuelve cada uno.

Clase	**Tipo**	**Responsabilidad**	**Atributos principales**	**Métodos principales**
Person	Abstracta	Datos comunes de cualquier persona del sistema	_full_name	full_name (propiedad que valida al asignar), description() abstracto, __str__()
User	Abstracta, hereda de Person	Cuenta de acceso al sistema: nombre de usuario, contraseña cifrada y permisos	_username, __password_hash, _salt	username (solo lectura), role (propiedad abstracta), permissions() abstracto, has_permission(), verify_password(), _compute_hash(), create() de clase, to_dict()
Administrator	Hereda de User	Usuario con acceso a todo, incluidos la cancelación, el deshacer, el historial de	Los heredados de User	Sobrescribe role, permissions() y description()

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
		acciones y la administración de usuarios		
Receptionist	Hereda de User	Usuario de recepción: registra pacientes, genera turnos, busca, consulta las colas y atiende	Los heredados de User	Sobrescribe role, permissions() y description()
Permission	Enumeración	Lista de permisos que se asignan a cada rol	REGISTER_PATIENT, UPDATE_PATIENT, CREATE_TICKET, SEARCH, VIEW_QUEUES, SERVE, VIEW_CARE_HISTORY, VIEW_STATS, CANCEL_TICKET, UNDO, VIEW_ACTION_LOG, MANAGE_USERS	Ninguno
Patient	Hereda de Person	Paciente registrado una sola vez, que puede tener varios turnos en el tiempo	_code, _age, _phone, _registered_at	code y registered_at (solo lectura), age y phone (validan al asignar), update_details(), description(), to_dict(), from_dict() de clase, __str__()

La contraseña se guarda como hash SHA-256 con una sal aleatoria por usuario, y create() genera esa sal al crear la cuenta. La función user_from_dict() lee un usuario del archivo y crea un Administrator o un Receptionist según el rol guardado.
update_details() valida el nombre, la edad y el teléfono antes de cambiar cualquiera, y devuelve los valores anteriores para poder deshacer la modificación. El motivo de la consulta y el tipo de

atención no se guardan en Patient, porque cambian en cada visita y pertenecen al turno.

Tipos de atención, turnos y registros de atención

El turno es el objeto central del sistema: guarda su paciente, su tipo de atención, el motivo y el estado. Su estado solo cambia por métodos que revisan si la transición está permitida.

Clase	**Tipo**	**Responsabilidad**	**Atributos principales**	**Métodos principales**
TicketStatus	Enumeración	Estados posibles de un turno	PENDING ("Pendiente"), IN_PROGRESS ("En atención"), SERVED ("Atendido"), CANCELLED ("Cancelado")	is_active (verdadera para Pendiente y En atención)
CareType	Abstracta	Define el nombre, el prefijo del número y si la atención es prioritaria	Ninguno propio	name, prefix e is_priority (propiedades abstractas), validate_for(patient), description(), to_dict()
NormalCare	Hereda de CareType	Atención normal, sin condiciones	Ninguno	Define name ("Normal"), prefix ("N") e is_priority (falso)
PriorityCare	Hereda de CareType	Atención prioritaria con el criterio que la justifica	_criterion; constantes CRITERIA y SENIOR_AGE = 60	criterion, validate_for() sobrescrito (Adulto mayor exige 60 años o más), description() sobrescrito ("Prioritaria (Embarazo)")
Ticket	Clase	Turno de un paciente en una visita	_number, _patient, _care_type, _reason, _created_at, _status, _served_at	Propiedades de solo lectura (number, status, sequence, is_active y las demás), start_care(),

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
				finish_care(), cancel(), revert_to(), to_dict(), from_dict() de clase, __str__()
CareRecord	Clase	Una fila del historial de pacientes atendidos	_ticket_number, _patient_code, _patient_name, _care_type, _registered_at, _served_at, _served_by	Propiedades de solo lectura, from_ticket() de clase, to_dict(), from_dict() de clase

Los criterios de PriorityCare son Adulto mayor, Embarazo, Discapacidad y Urgencia. La función make_care_type() crea el tipo correcto a partir del nombre guardado en el archivo.
start_care() pasa el turno de Pendiente a En atención, finish_care() de En atención a Atendido y cancel() de Pendiente a Cancelado. Cualquier otro cambio lanza ValidationError. revert_to() solo lo usan las acciones al deshacer.
sequence es la parte numérica del número (3 en P-003) y sirve para ordenar los turnos al reconstruir las colas. Ticket.to_dict() guarda el código del paciente y no el paciente completo.

Acciones del historial

Cada acción registrada es un objeto. Las acciones deshacibles saben revertir su propia operación, así que el historial llama a undo() sin preguntar de qué tipo es cada una.

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
Action	Abstracta	Cualquier acción del historial	_action_type, _description, _username, _data, _timestamp	Propiedades de solo lectura, is_undoable (propiedad abstracta)
LogOnlyAction	Hereda de Action	Acción que queda solo como evidencia: inicio y cierre de sesión, gestión de usuarios y acción deshecha	Los heredados	is_undoable (falso)
UndoableAction	Abstracta, hereda de Action	Acción que se puede revertir	_undone	is_undoable (verdadero),

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
		una sola vez		undone, undo(context), _revert(context) abstracto
RegisterPatientAction	Hereda de UndoableAction	Revierte el registro de un paciente	_patient, _ticket	_revert(): elimina al paciente y su primer turno, y saca el turno de la cola
CreateTicketAction	Hereda de UndoableAction	Revierte un turno nuevo de un paciente ya registrado	_ticket	_revert(): elimina el turno y lo saca de su cola
UpdatePatientAction	Hereda de UndoableAction	Revierte la modificación de un paciente	_patient, _previous_values	_revert(): restaura el nombre, la edad y el teléfono anteriores
CallNextAction	Hereda de UndoableAction	Revierte el llamado del siguiente paciente	_ticket	_revert(): devuelve el turno a Pendiente, al frente de su cola
FinishCareAction	Hereda de UndoableAction	Revierte el fin de una atención	_ticket, _record	_revert(): devuelve el turno a En atención y borra su registro de atendidos
CancelTicketAction	Hereda de UndoableAction	Revierte la cancelación de un turno	_ticket, _position	_revert(): devuelve el turno a Pendiente en la posición que tenía en la cola

undo() lanza NotUndoableError si la acción ya se deshizo; si no, llama a _revert() y marca la acción como deshecha. El context que recibe es cualquier objeto con los atributos patient_manager y ticket_manager. En el programa ese objeto es ClinicSystem, y en las pruebas puede ser uno más simple. Así actions.py no importa ClinicSystem y se evita una importación circular.

Los tipos de acción que ve el usuario quedan como constantes en el mismo módulo: Inicio de sesión, Cierre de sesión, Registro de paciente, Generación de turno, Modificación de paciente, Atención, Finalización de atención, Cancelación de turno, Gestión de usuarios y Acción

deshecha.

Estructuras de datos y persistencia

La cola y la pila son clases propias que envuelven una estructura de Python y lanzan un error claro cuando están vacías. El repositorio solo mueve listas de diccionarios entre la memoria y los archivos.

Clase	**Tipo**	**Responsabilidad**	**Atributos principales**	**Métodos principales**
TicketQueue	Estructura	Cola FIFO de turnos sobre collections.deque	_name, _items	enqueue(), dequeue(), peek(), is_empty(), count(), __len__(), list_all(), remove() (devuelve la posición que tenía el turno), insert_at(), push_front()
Stack	Estructura	Pila LIFO sobre una lista	_items	push(), pop(), peek(), is_empty(), count(), __len__(), list_from_top()
JsonRepository	Persistencia	Leer y escribir los archivos JSON en UTF-8 con manejo de errores	ENCODING, _folder	_path(), exists(), load(), save()

dequeue() y peek() lanzan EmptyQueueError con la cola vacía, y pop() y peek() de la pila lanzan EmptyStackError. insert_at() y push_front() solo se usan al deshacer una cancelación o un llamado.
Si un archivo no existe, load() lo crea con una lista vacía. Si está dañado, guarda una copia .corrupted y lanza CorruptedFileError. save() escribe primero en un archivo temporal y después reemplaza el original, para que una falla a mitad de la escritura no deje datos a medias.
Convertir cada objeto en diccionario es trabajo de su propio to_dict().

Servicios

Los servicios aplican las reglas del negocio sobre los modelos y las estructuras. ClinicSystem

los reúne y es la única clase que conoce la interfaz.

Clase	**Tipo**	**Responsabilidad**	**Atributos principales**	**Métodos principales**
UserManager	Servicio	Cargar, autenticar, crear y eliminar usuarios	_repository, _users; constante FILE = "users.json"	load(), login(), create_user(), delete_user(), list_all(), _create_default_users(), _save()
PatientManager	Servicio	Registrar, modificar, eliminar y consultar pacientes	_repository, _patients (código y su Patient); constante FILE = "patients.json"	load(), register(), get(), exists(), update(), delete(), next_code(), list_all(), patients_by_code(), count(), _save()
TicketManager	Servicio	Las dos colas, la generación de turnos, la atención, la cancelación y el historial de atendidos	_repository, _priority_queue, _normal_queue, _tickets, _care_records, _counter, _in_progress; constantes TICKETS_FILE y CARE_RECORDS_FILE	load(), create_ticket(), call_next(), finish_care(), cancel_ticket(), peek_next(), get(), get_status(), active_ticket_of(), pending_priority(), pending_normal(), priority_count(), normal_count(), queues_empty(), list_tickets(), list_care_records(), in_progress, _next_number(), _queue_for(), _save()
ActionHistory	Servicio	Registrar las acciones en dos pilas y deshacer la última deshacible	_history_stack, _undo_stack (dos Stack)	register(), last_action(), list_all(), has_actions(), can_undo(),

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
				undo_last()
SearchCriterion	Abstracta	Decidir si un turno coincide con el texto buscado	Ninguno	name (propiedad abstracta), matches(ticket, text) abstracto, _normalize(text) estático
ByPatientCode, ByPatientName, ByTicketNumber, ByStatus, ByCareType	Heredan de SearchCriterion	Cada una compara un dato distinto del turno. El código y el número buscan coincidencia exacta; el nombre acepta coincidencias parciales	Ninguno	Sobrescriben name y matches()
ClinicStats	Servicio	Calcular los conteos de las estadísticas	_patient_manager, _ticket_manager	compute()
ClinicSystem	Fachada	Unir los gestores, verificar permisos, registrar cada acción y guardar	_repository, _user_manager, patient_manager, ticket_manager, _history, _stats, _current_user, _warnings	login(), logout(), has_permission(), _require_permission(), register_patient(), create_ticket(), update_patient(), call_next(), finish_care(), cancel_ticket(), undo(), search(), stats(), create_user(), delete_user() y las consultas que usa la interfaz

TicketManager tiene además cuatro métodos que solo usan las acciones al deshacer: remove_pending_ticket(), return_to_front(), restore_cancelled() y revert_finish(). _queue_for() elige la cola con care_type.is_priority, sin preguntar la clase del tipo de atención.
ActionHistory usa dos pilas. La de historial guarda todas las acciones para mostrarlas; la de deshacer guarda solo las deshacibles. Así un inicio de sesión registrado después de una

cancelación no impide deshacer esa cancelación.
En search.py, la constante CRITERIA relaciona cada nombre visible ("Código de paciente", "Nombre", "Número de turno", "Estado", "Tipo de atención") con su criterio, y la función search() filtra los turnos con el criterio elegido. ClinicStats.compute() devuelve pacientes registrados, turnos pendientes, en atención, atendidos, cancelados, normales y prioritarios, y la cantidad en cada cola.
Cada operación de ClinicSystem que cambia datos sigue el mismo orden: exige el permiso, valida, ejecuta y registra la acción. Los permisos se revisan aquí y no solo en la interfaz, porque ocultar un botón no impide que otra parte del código llame a la función.

Utilidades

Validator revisa cada dato antes de que llegue a un modelo, y las excepciones propias llevan el mensaje en español que la interfaz muestra al usuario.

Clase	**Tipo**	**Responsabilidad**	**Atributos principales**	**Métodos principales**
Validator	Utilidad	Limpiar y validar cada dato de entrada con las reglas del proyecto	Expresiones regulares como constantes de clase	Métodos estáticos required(), patient_code(), full_name(), age(), phone(), reason(), username(), password()
ClinicError	Excepción base, hereda de Exception	Error propio del sistema, con un mensaje listo para mostrar	El mensaje	Los heredados de Exception

Cada método de Validator recibe el valor, lo limpia y lo devuelve, o lanza ValidationError. age() acepta texto y convierte el ValueError de int() en un ValidationError, así una edad con letras no cierra el programa.
Las hijas de ClinicError permiten que la interfaz atrape cualquier error del sistema con un solo bloque.

Excepción	**Cuándo se lanza**
ValidationError	Un dato no cumple su regla o un turno intenta un cambio de estado no permitido
DuplicateError	El código de paciente o el nombre de usuario ya existe
InvalidCredentialsError	El usuario o la contraseña no coinciden
PermissionDeniedError	El rol del usuario no tiene el permiso
NotFoundError	No existe el paciente, el turno o el usuario

Excepción	Cuándo se lanza
	pedido
EmptyQueueError	Se intenta sacar o consultar el frente de una cola vacía
EmptyStackError	Se intenta desapilar o consultar la cima de una pila vacía
NotUndoableError	La acción ya se deshizo
CorruptedFileError	Un archivo JSON no se puede leer
StorageError	Falla la escritura de un archivo

El módulo dates.py no tiene clases. Define el formato DATE_FORMAT (AAAA-MM-DD HH:MM:SS) y las funciones now_text() y parse_date().

Interfaz

La interfaz solo importa ClinicSystem, Permission y las excepciones. Nunca lee archivos ni toca las colas directamente.

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
App	Hereda de tk.Tk	Ventana raíz: crea ClinicSystem y cambia entre el inicio de sesión y la ventana principal	_system	show_login(), show_main(), report_callback_exception() sobrescrito
LoginView	Hereda de ttk.Frame	Formulario de inicio de sesión con la contraseña oculta	Campos de usuario y contraseña	Ingresar con el botón o con Enter; si falla, borra la contraseña y deja el foco en ella
MainView	Hereda de ttk.Frame	Ventana principal con menú, pestañas según los permisos y barra de estado	_notebook, _tabs	notify_change(), cerrar sesión con confirmación
BaseTab	Abstracta, hereda de ttk.Frame	Base común de todas las pestañas	title y required_permission (de clase),	refresh() abstracto, _run(operation,

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
			_system, _on_change	*args)
RegistrationTab	Hereda de BaseTab	Registrar pacientes, modificar sus datos y generarles un turno nuevo	Formulario, tabla de pacientes	refresh()
NewTicketDialog	Hereda de tk.Toplevel	Pedir motivo, tipo y criterio para el turno nuevo de un paciente registrado	Campos del turno	Generar turno y cerrar
TicketsTab	Hereda de BaseTab	Mostrar las dos colas, el siguiente y el turno en atención; atender, finalizar, cancelar y consultar el estado	Dos DataTable con su contador	refresh()
SearchTab	Hereda de BaseTab	Buscar turnos por el criterio elegido	Combobox de criterios, tabla de resultados	refresh()
CareHistoryTab	Hereda de BaseTab	Mostrar el historial de pacientes atendidos, del más reciente al más antiguo	Tabla de atendidos	refresh()
ActionsTab	Hereda de BaseTab	Mostrar la última acción y el historial, y deshacer la última acción	required_permission = VIEW_ACTION_LOG	refresh()
StatsTab	Hereda de BaseTab	Mostrar las estadísticas	Una etiqueta por valor	refresh()
UsersTab	Hereda de	Crear y eliminar	required_permiss	refresh()

Clase	Tipo	Responsabilidad	Atributos principales	Métodos principales
	BaseTab	usuarios	ion = MANAGE_USERS	
DataTable	Hereda de ttk.Frame	Tabla reutilizable sobre ttk.Treeview con barra de desplazamiento	El Treeview y sus columnas	load(rows), clear(), selected_id()

_run() llama a una operación de ClinicSystem, atrapa cualquier ClinicError y muestra su mensaje; si todo sale bien, pide a MainView que refresque todas las pestañas. Así ningún botón repite su propio try y except. MainView crea solo las pestañas cuyo required_permission tiene el usuario, de modo que el recepcionista no ve Historial de acciones ni Usuarios.
El módulo widgets.py incluye además las funciones show_error(), show_info() y confirm(), que muestran los mensajes de error, de información y de confirmación.

Conceptos de POO

Cada concepto que pide el enunciado aparece en al menos una clase de la propuesta.

Concepto	Dónde se aplica
Constructor __init__	Todas las clases
Encapsulamiento y @property	Patient.age y Patient.phone validan al asignar. User.username y Ticket.number son de solo lectura. Ticket.status solo cambia con métodos que revisan la transición
Atributos y métodos protegidos o privados	User.__password_hash, TicketQueue._items, TicketManager._priority_queue, ClinicSystem._require_permission(), JsonRepository._path()
Herencia	Administrator y Receptionist heredan de User, que hereda de Person. Patient hereda de Person. También forman jerarquías los tipos de atención, las acciones, los criterios de búsqueda, las excepciones y las pestañas
Clases abstractas	Person, User, CareType, Action, UndoableAction, SearchCriterion, BaseTab
Polimorfismo	user.permissions() responde según el rol. care_type.is_priority elige la cola. action.undo() revierte cada operación a su manera. criterion.matches() filtra según el

Concepto	Dónde se aplica
	criterio. tab.refresh() actualiza cada pestaña con su contenido
Sobrescritura	description() en Administrator, Receptionist y Patient. validate_for() en PriorityCare. __str__() en Ticket y Patient
Composición	Ticket contiene su CareType. TicketManager contiene dos TicketQueue. ActionHistory contiene dos Stack. ClinicSystem contiene el repositorio y los gestores. MainView contiene sus pestañas
Agregación	Ticket referencia a un Patient que existe por su cuenta
Separación entre lógica e interfaz	El paquete ui solo importa ClinicSystem, Permission y las excepciones

Cobertura del enunciado

Todo lo que el enunciado pide representar tiene al menos una clase, y el programa queda repartido en seis paquetes y varios módulos.

El enunciado pide representar	Clases
Personas que usan el sistema	Person, User, Administrator, Receptionist, Permission
Pacientes	Patient, PatientManager
Turnos	Ticket, TicketStatus, CareRecord
Tipos de atención	CareType, NormalCare, PriorityCare
Acciones del historial	Action, LogOnlyAction, UndoableAction y sus seis hijas
Administración de las colas	TicketQueue, TicketManager
Administración de la pila	Stack, ActionHistory
Manejo de archivos	JsonRepository
Interfaz	App, LoginView, MainView, BaseTab y sus pestañas, NewTicketDialog, DataTable

Validaciones y manejo de excepciones

Dónde se valida y qué pasa con un error

El sistema revisa cada dato antes de guardarlo. Si un dato no cumple su regla, la operación se detiene, aparece un mensaje en español que dice qué corregir y el programa sigue abierto (RNF-10). El formulario conserva lo que se escribió, así que basta con corregir el campo señalado.
Las validaciones están en la lógica del sistema. La clase Validator revisa los formatos (código, nombre, edad, teléfono, motivo, nombre de usuario y contraseña), y ClinicSystem revisa los permisos y las reglas del negocio antes de ejecutar cada operación. La interfaz además deshabilita los botones que el rol no puede usar, pero si alguna parte del código llamara a una función restringida, la lógica la rechazaría igual (RNF-08).
Cuando una validación falla, se lanza una excepción propia que ya trae el mensaje para el usuario. Todas heredan de ClinicError, y la interfaz las atrapa en un solo lugar para mostrar ese mensaje.

Validaciones que pide el enunciado

Cada una de las 13 validaciones del punto 10 tiene una regla concreta, un mensaje y la excepción que la comunica. Los códigos y números de turno de los mensajes son ejemplos: el sistema pone los reales.

N.º	**Validación**	**Regla que aplica el sistema**	**Mensaje al usuario**	**Excepción**
1	Campos obligatorios	En el registro: código, nombre, edad, teléfono, motivo y tipo de atención. En el inicio de sesión: usuario y contraseña. Al crear un usuario: nombre completo, usuario, contraseña y rol. Un campo con solo espacios cuenta como vacío	"Completa el campo Teléfono." (con el nombre del campo vacío)	ValidationError
2	Códigos repetidos	No existe otro paciente con el	"Ya existe un paciente con el	DuplicateError

N.º	Validación	Regla que aplica el sistema	Mensaje al usuario	Excepción
		mismo código (RN-03)	código PAC-0007."	
3	Números de turno repetidos	El contador global no repite números (RN-04). Igual se revisa al generar un turno y al cargar tickets.json, por si alguien editó el archivo a mano	"El turno N-004 ya existe. Revisa tickets.json."	ValidationError
4	Edad válida	Número entero entre 0 y 120. Letras, decimales o negativos dan el mismo mensaje	"La edad debe ser un número entero entre 0 y 120."	ValueError, relanzada como ValidationError
5	Formato del teléfono	8 dígitos que empiezan con 2, 3, 4, 5, 6 o 7	"El teléfono debe tener 8 dígitos y empezar con 2 a 7, por ejemplo 55551234."	ValidationError
6	Tipo de atención seleccionado	Hay un tipo elegido. Si es Prioritaria, también un criterio (RF-13)	"Selecciona el tipo de atención." o "Selecciona el criterio de atención prioritaria."	ValidationError
7	Pacientes antes de atender	Hay al menos un turno Pendiente en alguna de las dos colas	"No hay pacientes en espera."	EmptyQueueError
8	Acciones antes de deshacer	La pila de deshacer tiene al menos una acción	"No hay acciones que se puedan deshacer."	EmptyStackError
9	Selección antes de modificar o	Hay una fila seleccionada	"Selecciona un paciente en la	La revisa la interfaz. Si

N.º	Validación	Regla que aplica el sistema	Mensaje al usuario	Excepción
	cancelar	antes de modificar un paciente, generarle un turno, cancelar un turno o eliminar un usuario	tabla.", "Selecciona un turno en la tabla." o "Selecciona un usuario en la tabla."	llegara un código inexistente a la lógica, esta lanza NotFoundError
10	Credenciales	El usuario existe y la contraseña coincide con el hash guardado. El mensaje no dice cuál de los dos falló (RNF-09)	"Usuario o contraseña incorrectos."	InvalidCredentialsError
11	Permisos	El rol del usuario tiene el permiso de la operación (RN-10)	"Tu usuario no tiene permiso para esta acción."	PermissionDeniedError
12	Archivos antes de leerlos	Antes de leer un JSON se comprueba que exista. Si falta, se crea con una lista vacía, y si falta users.json se crean los usuarios iniciales (RF-42)	Ninguno: el sistema sigue con normalidad	FileNotFoundError, atrapada dentro de JsonRepository
13	Confirmación antes de cancelar o eliminar	Cancelar un turno, eliminar un usuario, deshacer, cerrar sesión y salir piden confirmación (RNF-04)	Ver la sección de mensajes de confirmación	No aplica

Validaciones adicionales

Los formatos de los datos y las reglas del negocio suman otras validaciones. El enunciado no las nombra una por una, pero sin ellas el sistema guardaría datos que rompen esas reglas.

Validación	Regla	Mensaje al usuario	Excepción
Formato del código	PAC, un guion y cuatro dígitos. Se guarda en mayúsculas	"El código debe tener el formato PAC-0000."	ValidationError
Nombre del paciente	Entre 3 y 80 caracteres, solo letras (con tildes y ñ), espacios y guiones. El guion va entre dos palabras, como en López-Pérez	"El nombre solo puede tener letras, espacios y guiones entre palabras." o "El nombre debe tener entre 3 y 80 caracteres."	ValidationError
Motivo de la consulta	Entre 5 y 200 caracteres	"Describe el motivo con al menos 5 caracteres." o "El motivo no puede pasar de 200 caracteres."	ValidationError
Criterio Adulto mayor	El paciente tiene 60 años o más (RN-09)	"El criterio Adulto mayor requiere 60 años o más."	ValidationError
Turno activo	El paciente no tiene otro turno Pendiente o En atención (RN-07)	"Este paciente ya tiene el turno N-004 activo."	ValidationError
Atención en curso	No hay otro turno En atención al llamar al siguiente (RN-08)	"Finaliza la atención de N-002 antes de llamar al siguiente."	ValidationError
Estado al cancelar	El turno está Pendiente (RN-05)	"Solo se pueden cancelar turnos pendientes."	ValidationError
Turno consultado	El número existe al consultar su estado (RF-18)	"No existe el turno N-050."	NotFoundError
Nombre de usuario	Entre 4 y 20 caracteres: letras, números o guion bajo	"El nombre de usuario debe tener entre 4 y 20 caracteres, con letras, números o	ValidationError

Validación	Regla	Mensaje al usuario	Excepción
		guion bajo."	
Usuario repetido	No existe otro usuario con ese nombre	"Ese nombre de usuario ya existe."	DuplicateError
Contraseña	Al menos 6 caracteres	"La contraseña debe tener al menos 6 caracteres."	ValidationError
Eliminar usuario	Nadie elimina su propia cuenta y siempre queda un administrador (RN-12)	"No puedes eliminar tu propia cuenta." o "Debe quedar al menos un administrador."	ValidationError

Mensajes de confirmación

Cinco operaciones preguntan antes de ejecutarse. Si el usuario responde No, nada cambia.

Operación	Mensaje	Al confirmar
Cancelar un turno	"¿Cancelar el turno N-004? Esta acción se puede deshacer."	El turno pasa a Cancelado y sale de su cola
Eliminar un usuario	"¿Eliminar al usuario recepcion? Esta acción no se puede deshacer."	El usuario se borra de users.json
Deshacer	"¿Deshacer la última acción: Cancelación de turno N-004?"	Los datos vuelven al estado anterior a esa acción
Cerrar sesión	"¿Cerrar la sesión?"	Vuelve a la ventana de inicio de sesión
Salir, con el botón o con la X de la ventana	"¿Salir del sistema?"	El programa se cierra. Los datos ya están guardados, porque se guardan después de cada operación

Manejo de excepciones

Ninguna de estas situaciones cierra el programa. Cada una se atrapa y termina en un mensaje comprensible, y los detalles técnicos quedan en errors.log (RNF-11).

Situación	Excepción	Respuesta del sistema
Letras o decimales en la edad	ValueError al convertir, relanzada como ValidationError	Mensaje de edad inválida. El formulario conserva los datos

Situación	Excepción	Respuesta del sistema
Archivo inexistente	FileNotFoundError dentro de JsonRepository	Se crea el archivo con una lista vacía y el sistema sigue
Archivo dañado	json.JSONDecodeError o UnicodeDecodeError (el archivo no está en UTF-8), relanzadas como CorruptedFileError	Se guarda una copia .corrupted con fecha y hora, se avisa al iniciar y esa lista empieza vacía (RNF-12)
Error al guardar	OSError, relanzada como StorageError	Aviso al usuario. Los datos en memoria se conservan y el archivo anterior queda intacto, porque primero se escribe un temporal (RNF-13)
Atender con las dos colas vacías	EmptyQueueError	"No hay pacientes en espera."
Deshacer con la pila vacía	EmptyStackError	"No hay acciones que se puedan deshacer."
Operación sin permiso	PermissionDeniedError	"Tu usuario no tiene permiso para esta acción."
Error inesperado	Cualquier Exception, atrapada en report_callback_exception de Tkinter	"Ocurrió un error inesperado. La operación no se completó." y el detalle en errors.log

Las excepciones propias forman una jerarquía de dos niveles. ClinicError hereda de Exception, y las diez excepciones del sistema (ValidationError, DuplicateError, InvalidCredentialsError, PermissionDeniedError, NotFoundError, EmptyQueueError, EmptyStackError, NotUndoableError, CorruptedFileError y StorageError) heredan directamente de ClinicError. Cada una trae su mensaje en español. La interfaz atrapa ClinicError en un solo método de la pestaña base, así no repite un try/except en cada botón.

Verificación contra el enunciado

Las 13 validaciones del punto 10 y las seis situaciones del punto 11 están cubiertas, y todas menos una tienen un caso de prueba en la sección siguiente.

Punto 10 del enunciado	Dónde se cubre	Caso de prueba
Campos obligatorios	Validación 1	CP-03, CP-44
Códigos repetidos	Validación 2	CP-06
Números de turno repetidos	Validación 3	CP-69
Edad válida	Validación 4	CP-07, CP-08
Formato del teléfono	Validación 5	CP-09

Punto 10 del enunciado	**Dónde se cubre**	**Caso de prueba**
Tipo de atención seleccionado	Validación 6	CP-10, CP-39
Pacientes antes de atender	Validación 7	CP-14
Acciones antes de deshacer	Validación 8	CP-22
Selección antes de modificar o cancelar	Validación 9	CP-47
Credenciales	Validación 10	CP-02, CP-29
Permisos	Validación 11	CP-19
Archivos antes de leerlos	Validación 12	CP-28
Confirmación antes de cancelar o eliminar	Validación 13 y mensajes de confirmación	CP-17, CP-52
Una entrada incorrecta no cierra el programa	RNF-10 y la interfaz atrapa toda ClinicError	CP-07

Punto 11 del enunciado	**Excepción**	**Caso de prueba**
Datos numéricos incorrectos	ValidationError (a partir del ValueError)	CP-07
Archivos inexistentes	FileNotFoundError	CP-28
Archivos dañados	CorruptedFileError	CP-27, CP-73
Errores al guardar	StorageError	CP-70
Operaciones sobre pilas o colas vacías	EmptyStackError, EmptyQueueError	CP-22, CP-14
Errores inesperados	Cualquier Exception, en report_callback_exception	Fase de pruebas

Los errores inesperados no se pueden provocar desde la interfaz sin cambiar el código. Se comprueban en la fase de pruebas lanzando una excepción temporal en un botón y revisando que el detalle se escriba en errors.log.

Casos de prueba

Cómo se ejecutan

Hay 72 casos de prueba, y cada uno de los 44 requerimientos funcionales aparece en al menos uno. La tabla de cobertura del final muestra qué casos revisan cada requerimiento.
Salvo que la columna Preparación diga otra cosa, cada caso empieza con la carpeta data vacía y la sesión iniciada como admin. Con la carpeta vacía, el sistema crea los usuarios admin (contraseña Admin123) y recepcion (contraseña Recep123). El contador de turnos es global, así que los números que aparecen en cada caso suponen que se partió de cero.
Los casos están agrupados por módulo. Dentro de cada grupo los números no van seguidos, porque CP-01 a CP-28 se definieron primero y los casos nuevos empiezan en CP-29.
En la fase de pruebas se agregan a cada tabla las columnas de resultado obtenido, estado (Pasó o Falló) y captura.

Inicio de sesión y usuarios

Doce casos revisan el acceso, los permisos de cada rol y la administración de usuarios.

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-01	RF-01, RF-02	Ninguna	Ingresar admin y Admin123 y pulsar Ingresar	Se abre la ventana principal con todas las pestañas. La barra de estado muestra el usuario y el rol Administrador
CP-02	RF-02, RNF-09	Ninguna	Ingresar admin y clave	Aparece "Usuario o contraseña incorrectos.", la contraseña se borra y la ventana de inicio sigue abierta
CP-03	RF-01	Ninguna	Pulsar Ingresar con los dos campos vacíos	Aparece "Completa el campo Usuario." y no se entra
CP-04	RF-03	Ninguna	Ingresar recepcion y Recep123	No aparecen las pestañas Historial de acciones ni

ID	Requisito	Preparación	Acción y datos	Resultado esperado
				Usuarios. El botón Cancelar turno está deshabilitado
CP-29	RF-02, RNF-09	Ninguna	Ingresar pedro, que no existe, y Admin123	Aparece el mismo mensaje de CP-02, sin indicar si falló el usuario o la contraseña
CP-30	RF-04	Sesión de admin	Archivo, Cerrar sesión y confirmar. Entrar otra vez como admin y abrir Historial de acciones	Vuelve a la ventana de inicio. El historial muestra un "Cierre de sesión" del usuario admin
CP-31	RF-05, RNF-04	Sesión iniciada	Archivo, Salir y responder No. Repetir y responder Sí. Repetir con la X de la ventana	Con No la aplicación sigue abierta y con Sí se cierra. La X pide la misma confirmación
CP-32	RF-06, RF-08	Sesión de admin	En Usuarios, crear a Carla Méndez, cmendez, Clave123, Recepcionista. Cerrar sesión y entrar con cmendez	La tabla de usuarios muestra a Carla Méndez con el rol Recepcionista. cmendez entra con los mismos permisos de CP-04
CP-33	RF-06	Existe cmendez	Crear otro usuario con el nombre cmendez	Aparece "Ese nombre de usuario ya existe." y no se crea nada
CP-34	RF-06, RF-43	Sesión de admin	Crear un usuario con la contraseña abc	Aparece "La contraseña debe tener al menos 6 caracteres." y el

ID	Requisito	Preparación	Acción y datos	Resultado esperado
				usuario no se crea
CP-35	RF-07, RNF-04	Existe cmendez	Seleccionar cmendez, pulsar Eliminar seleccionado y confirmar. Intentar entrar con cmendez	El usuario sale de la tabla y ya no puede iniciar sesión
CP-36	RF-07, RN-12	Sesión de admin	Seleccionar la cuenta admin y pulsar Eliminar seleccionado	Aparece "No puedes eliminar tu propia cuenta." La cuenta sigue en la tabla

Pacientes

Veintidós casos cubren el registro, sus validaciones, la modificación y el turno nuevo para un paciente que regresa. En los casos de validación, el resto del formulario lleva los datos válidos de CP-05.

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-05	RF-09, RF-16, RF-20	Sin pacientes	Registrar PAC-0001, Ana López, 30, 55551234, "Dolor de cabeza", Normal	El paciente queda registrado y se genera el turno N-001, Pendiente, al final de la cola normal. Un mensaje muestra el número
CP-06	RF-12	Existe PAC-0001	Registrar otro paciente con el código PAC-0001	Aparece "Ya existe un paciente con el código PAC-0001." No se guarda el paciente ni el turno
CP-07	RF-09, RF-43, RNF-10	Ninguna	Registrar con la edad "abc"	Aparece "La edad debe ser un

ID	Requisito	Preparación	Acción y datos	Resultado esperado
				número entero entre 0 y 120." El programa sigue abierto y el formulario conserva los datos
CP-08	RF-09, RF-43	Ninguna	Registrar con la edad 150	El mismo mensaje de CP-07
CP-09	RF-09, RF-43	Ninguna	Registrar con el teléfono 1234	Aparece "El teléfono debe tener 8 dígitos y empezar con 2 a 7, por ejemplo 55551234."
CP-10	RF-09	Ninguna	Registrar sin elegir tipo de atención	Aparece "Selecciona el tipo de atención."
CP-11	RF-13, RN-09	Ninguna	Registrar como Prioritaria, criterio Adulto mayor, edad 30	Aparece "El criterio Adulto mayor requiere 60 años o más."
CP-12	RF-17, RN-07	Ana López tiene N-001 Pendiente	Seleccionar a Ana, pulsar Nuevo turno, motivo "Control de presión", Normal	Aparece "Este paciente ya tiene el turno N-001 activo." No se crea el turno
CP-37	RF-10, RF-15	Ana López registrada como en CP-05	Revisar la tabla de pacientes	Ana aparece con su código, nombre, edad, teléfono y la fecha y hora del registro en formato AAAA-MM-DD HH:MM:SS. Nadie escribió esa fecha

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-38	RF-11	Existen PAC-0001 y PAC-0002	Pulsar Limpiar en el formulario	El campo de código propone PAC-0003
CP-39	RF-13	Ninguna	Elegir Normal y después Prioritaria. Registrar sin criterio	Con Normal el cuadro de criterio está deshabilitado y con Prioritaria se habilita. Sin criterio aparece "Selecciona el criterio de atención prioritaria." y no se registra nada
CP-40	RF-13, RN-09	Sin pacientes	Registrar PAC-0001, Luis Pérez, 67, 22334455, "Control de diabetes", Prioritaria, Adulto mayor	Se genera P-001, Pendiente, al final de la cola prioritaria
CP-41	RF-09, RF-43	Ninguna	Registrar con el nombre "Ana123"	Aparece "El nombre solo puede tener letras, espacios y guiones entre palabras."
CP-42	RF-09, RF-43	Ninguna	Registrar con el motivo "Tos"	Aparece "Describe el motivo con al menos 5 caracteres."
CP-43	RF-09, RF-43	Ninguna	Registrar con el código "PAC-12"	Aparece "El código debe tener el formato PAC-0000."
CP-44	RF-09, RF-43	Ninguna	Registrar con el teléfono vacío	Aparece "Completa el campo Teléfono."

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-45	RF-14	Ana López registrada	Seleccionar a Ana, pulsar Modificar seleccionado, cambiar la edad a 31 y el teléfono a 44441234, pulsar Guardar cambios	La tabla muestra los valores nuevos. El campo de código no se puede editar
CP-46	RF-14, RNF-10	Ana López registrada	Modificar con el teléfono 44441234 y la edad -5, pulsar Guardar cambios	Aparece el mensaje de edad. Ana conserva la edad 30 y el teléfono 55551234, aunque el teléfono nuevo era válido
CP-47	Validación de selección	Hay pacientes y ninguna fila seleccionada	Pulsar Modificar seleccionado y después Nuevo turno	En los dos casos aparece "Selecciona un paciente en la tabla."
CP-48	RF-17	Ana López tiene N-001 Atendido	Seleccionar a Ana, pulsar Nuevo turno, motivo "Control de presión", Normal	Se crea N-002, Pendiente, al final de la cola normal. Ana sigue siendo una sola fila en la tabla de pacientes
CP-71	RF-09, RF-43	Sin pacientes	Registrar PAC-0001, Ana López-Pérez, 30, 55551234, "Dolor de cabeza", Normal	El paciente queda registrado con el nombre "Ana López-Pérez" y se genera N-001
CP-72	RF-09, RF-43	Ninguna	Registrar con el nombre "-Ana López" y después con	En los dos casos aparece "El nombre solo puede tener

ID	Requisito	Preparación	Acción y datos	Resultado esperado
			"Ana López--Pérez"	letras, espacios y guiones entre palabras." y no se registra nada

Turnos y colas

Once casos revisan el orden de atención, los estados del turno, la cancelación y lo que muestra la pestaña de colas.

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-13	RF-23, RN-01, RN-02	Colas vacías	Registrar dos pacientes normales (N-001 y N-002) y uno prioritario (P-003). Pulsar Atender siguiente y Finalizar atención tres veces	Se atienden en este orden: P-003, N-001, N-002
CP-14	RF-26	Colas vacías	Pulsar Atender siguiente	Aparece "No hay pacientes en espera." La pestaña avisa que las dos colas están vacías
CP-15	RN-08	N-001 En atención y N-002 Pendiente	Pulsar Atender siguiente	Aparece "Finaliza la atención de N-001 antes de llamar al siguiente." N-002 sigue en la cola
CP-16	RF-24, RF-35	N-001 En atención	Pulsar Finalizar atención	N-001 queda Atendido y el recuadro En atención se vacía. El historial de atendidos muestra N-001 con la fecha y hora de atención

ID	Requisito	Preparación	Acción y datos	Resultado esperado
				y el usuario admin
CP-17	RF-19, RF-25	N-001 y N-002 Pendientes en la cola normal	Seleccionar N-002, pulsar Cancelar turno y confirmar	N-002 queda Cancelado y sale de la cola normal. El contador de esa cola baja de 2 a 1
CP-18	RN-05	N-001 Atendido	Intentar cancelar N-001. En la interfaz no se puede seleccionar porque las tablas de colas solo muestran pendientes, así que se prueba con la prueba automática del gestor de turnos	Aparece "Solo se pueden cancelar turnos pendientes." y N-001 sigue Atendido
CP-19	RN-10	Sesión de recepcion con N-001 Pendiente	Seleccionar N-001 y revisar el botón Cancelar turno	El botón está deshabilitado. La prueba automática de la lógica confirma que una llamada directa se rechaza con "Tu usuario no tiene permiso para esta acción."
CP-49	RF-20, RF-21, RF-25, RN-04	Colas vacías	Registrar cuatro pacientes en este orden: normal, prioritario, normal, prioritario	La cola prioritaria muestra P-002 y P-004 con contador 2. La normal muestra N-001 y N-003 con contador 2. Ningún número se repite entre

ID	Requisito	Preparación	Acción y datos	Resultado esperado
				las colas
CP-50	RF-22	Estado final de CP-49	Revisar el recuadro Siguiente. Atender y finalizar dos veces y revisarlo de nuevo	Primero muestra P-002. Después de atender P-002 y P-004 muestra N-001
CP-51	RF-18	N-001 Pendiente y P-002 Atendido	Consultar el estado de N-001, después el de P-002 y después el de X-999	Muestra Pendiente y luego Atendido. Para X-999 aparece "No existe el turno X-999."
CP-52	RNF-04	Sesión de admin con N-001 Pendiente, al menos una acción deshacible y el usuario cmendez creado	Pulsar Cancelar turno, Deshacer última acción y Eliminar seleccionado, y responder No en cada confirmación	Cada operación pide confirmación, por ejemplo "¿Cancelar el turno N-001? Esta acción se puede deshacer." Con No, ninguna se ejecuta y los datos no cambian

Historial de acciones y deshacer

Nueve casos revisan lo que guarda el historial y que cada acción deshacible vuelva los datos a su estado anterior. Todos se hacen con la sesión de admin y confirmando el deshacer.

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-20	RF-30	N-001, N-002 y N-003 Pendientes en la cola normal	Cancelar N-002 y confirmar. Pulsar Deshacer última acción	N-002 vuelve a Pendiente en la segunda posición: N-001, N-002, N-003
CP-21	RF-30	N-001 al frente de la cola normal	Pulsar Atender siguiente y	N-001 vuelve a Pendiente al

ID	Requisito	Preparación	Acción y datos	Resultado esperado
			después Deshacer última acción	frente de su cola y el recuadro En atención queda vacío
CP-22	RF-31	Sesión recién iniciada, sin otras acciones	Pulsar Deshacer última acción	Aparece "No hay acciones que se puedan deshacer."
CP-53	RF-27, RF-28, RF-29	Colas vacías	Registrar a Ana López, atender y finalizar su turno. Abrir Historial de acciones	La última acción es "Finalización de atención". La tabla muestra de arriba abajo: Finalización de atención, Atención, Registro de paciente e Inicio de sesión. Cada fila trae fecha y hora, tipo, una descripción con el turno o el paciente, el usuario admin y si se puede deshacer
CP-54	RF-30	Ana López recién registrada con N-001	Pulsar Deshacer última acción	Ana sale de la tabla de pacientes y N-001 sale de la cola normal. En el historial, el registro queda marcado como deshecho y encima aparece una "Acción deshecha"
CP-55	RN-04	Estado final de CP-54	Registrar a otro paciente con atención Normal	El turno nuevo es N-002. El número N-001 no se

ID	Requisito	Preparación	Acción y datos	Resultado esperado
				vuelve a usar
CP-56	RF-30	Ana López modificada como en CP-45	Pulsar Deshacer última acción	Ana vuelve a tener la edad 30 y el teléfono 55551234
CP-57	RF-30	N-001 recién finalizado	Pulsar Deshacer última acción	N-001 vuelve a En atención y su fila desaparece del historial de atendidos
CP-58	RF-30, RN-11	admin cancela N-002, cierra sesión y vuelve a entrar como admin	Pulsar Deshacer última acción	Se deshace la cancelación de N-002 aunque después hubo un cierre y un inicio de sesión. Esas dos acciones siguen en el historial como evidencia

Búsquedas, historial de atendidos y estadísticas

Once casos revisan los cinco criterios de búsqueda, el orden del historial de atendidos y que las estadísticas se actualicen solas.
Los casos de búsqueda usan estos datos: PAC-0001 María Gómez con N-001 Atendido, PAC-0002 Marco Ruiz con N-002 Pendiente y PAC-0003 Luis Pérez con P-003 Pendiente.

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-23	RF-32, RF-33	Datos de búsqueda	Criterio Nombre, texto "mar"	Aparecen N-001 de María Gómez y N-002 de Marco Ruiz
CP-24	RF-34	Datos de búsqueda	Criterio Código de paciente, texto PAC-9999	Un mensaje avisa que no hay resultados y la tabla queda vacía
CP-59	RF-32	Datos de búsqueda	Criterio Código de paciente,	Aparece solo P-003 de Luis

ID	Requisito	Preparación	Acción y datos	Resultado esperado
			texto PAC-0003	Pérez
CP-60	RF-32	Datos de búsqueda	Criterio Número de turno, texto N-002	Aparece solo N-002
CP-61	RF-32	Datos de búsqueda	Criterio Estado, texto "Pendiente"	Aparecen N-002 y P-003
CP-62	RF-32	Datos de búsqueda	Criterio Tipo de atención, texto "Prioritaria"	Aparece solo P-003
CP-63	RF-33	Datos de búsqueda	Criterio Nombre, texto "MARIA"	Aparece N-001 de María Gómez, aunque el texto va en mayúsculas y sin tilde
CP-64	Validación de campo obligatorio	Datos de búsqueda	Criterio Nombre, texto vacío	Un mensaje pide escribir un texto y no se busca nada
CP-65	RF-35, RF-36	Colas vacías	Registrar dos pacientes normales. Atender y finalizar N-001 y después N-002. Abrir Historial de atendidos	N-002 aparece arriba y N-001 abajo. Cada fila tiene turno, paciente, tipo, fecha de registro, fecha de atención y usuario
CP-25	RF-37, RF-38, RF-39	Colas vacías	Registrar dos pacientes normales y uno prioritario. Atender y finalizar uno. Abrir Estadísticas	Registrados 3, pendientes 2, en atención 0, atendidos 1, cancelados 0, normales 2, prioritarios 1, cola prioritaria 0 y cola normal 2
CP-66	RF-37, RF-39	Estado final de CP-25	Cancelar N-002 y volver a	Pendientes 1, cancelados 1 y

ID	Requisito	Preparación	Acción y datos	Resultado esperado
			Estadísticas sin reiniciar la aplicación	cola normal 1. Registrados sigue en 3 y normales en 2, porque se cuentan los turnos en cualquier estado

Almacenamiento y archivos

Siete casos revisan que los datos se guarden después de cada operación y que un archivo faltante, dañado o bloqueado no cierre el programa. Los archivos se modifican con la aplicación cerrada, salvo en CP-67.

ID	Requisito	Preparación	Acción y datos	Resultado esperado
CP-26	RF-41, RNF-16	Pacientes registrados y turnos en las dos colas	Cerrar la aplicación y abrirla de nuevo	Pacientes, turnos y colas están iguales y en el mismo orden
CP-27	RNF-12	Escribir texto inválido dentro de tickets.json	Abrir la aplicación	Aparece un aviso del archivo dañado, se crea una copia tickets.corrupted-AAAAMMDD-HHMMSS.json y la aplicación abre con la lista de turnos vacía
CP-28	RF-42	Borrar users.json	Abrir la aplicación	Se crean admin y recepcion, y los dos pueden entrar con sus contraseñas
CP-67	RF-40, RNF-17	Sesión de admin	Registrar a Ana López y, sin cerrar la aplicación, abrir data/patients.json y data/tickets.json	Ana y N-001 ya están en los archivos. "López" se lee con su tilde y el contenido tiene sangría. El Bloc

ID	Requisito	Preparación	Acción y datos	Resultado esperado
			en el Bloc de notas	de notas muestra UTF-8 en la barra de estado
CP-68	RF-41	N-001 En atención	Cerrar la aplicación y abrirla de nuevo	N-001 sigue en el recuadro En atención y Finalizar atención funciona
CP-69	RF-44	tickets.json tiene el turno N-004. Copiar su bloque para que aparezca dos veces	Abrir la aplicación	Aparece "El turno N-004 ya existe. Revisa tickets.json." y la aplicación no se cierra sin aviso
CP-70	RNF-21	Marcar data/tickets.json como de solo lectura en Windows	Abrir la aplicación y registrar un paciente	Un aviso dice que no se pudo guardar. La aplicación sigue abierta y el turno nuevo sigue en su cola mientras no se cierre
CP-73	RNF-12, RNF-17	Ana López ya fue atendida. Abrir data/care_records.json en el Bloc de notas y guardarlo con la codificación ANSI	Abrir la aplicación	Aparece un aviso del archivo dañado, se crea una copia care_records.corrupted-AAAAMMDD-HHMMSS.json y la aplicación abre con el historial de atendidos vacío

Cobertura de los requerimientos funcionales

Los 44 requerimientos funcionales tienen al menos un caso.

RF	Qué pide	Casos
RF-01	Iniciar sesión	CP-01, CP-03
RF-02	Verificar credenciales	CP-01, CP-02, CP-29

RF	Qué pide	Casos
RF-03	Funciones según el rol	CP-04, CP-19
RF-04	Cerrar sesión	CP-30
RF-05	Salir con confirmación	CP-31
RF-06	Crear usuarios	CP-32, CP-33, CP-34
RF-07	Eliminar usuarios	CP-35, CP-36
RF-08	Listar usuarios	CP-32
RF-09	Registrar paciente	CP-05, CP-07 a CP-11, CP-41 a CP-44, CP-71, CP-72
RF-10	Fecha de registro automática	CP-37
RF-11	Proponer el siguiente código	CP-38
RF-12	Rechazar código repetido	CP-06
RF-13	Pedir criterio de prioridad	CP-11, CP-39, CP-40
RF-14	Modificar paciente	CP-45, CP-46
RF-15	Listar pacientes	CP-37
RF-16	Generar turno al registrar	CP-05
RF-17	Turno nuevo para paciente registrado	CP-12, CP-48
RF-18	Consultar estado de un turno	CP-51
RF-19	Cancelar turno pendiente	CP-17, CP-18, CP-19, CP-20, CP-52
RF-20	Encolar según el tipo	CP-05, CP-49
RF-21	Mostrar pendientes en orden	CP-49
RF-22	Mostrar el siguiente	CP-50
RF-23	Atender prioritarios primero	CP-13
RF-24	Finalizar atención	CP-16
RF-25	Cantidad en cada cola	CP-17, CP-49
RF-26	Avisar colas vacías	CP-14
RF-27	Registrar cada acción	CP-30, CP-53
RF-28	Mostrar la última acción	CP-53

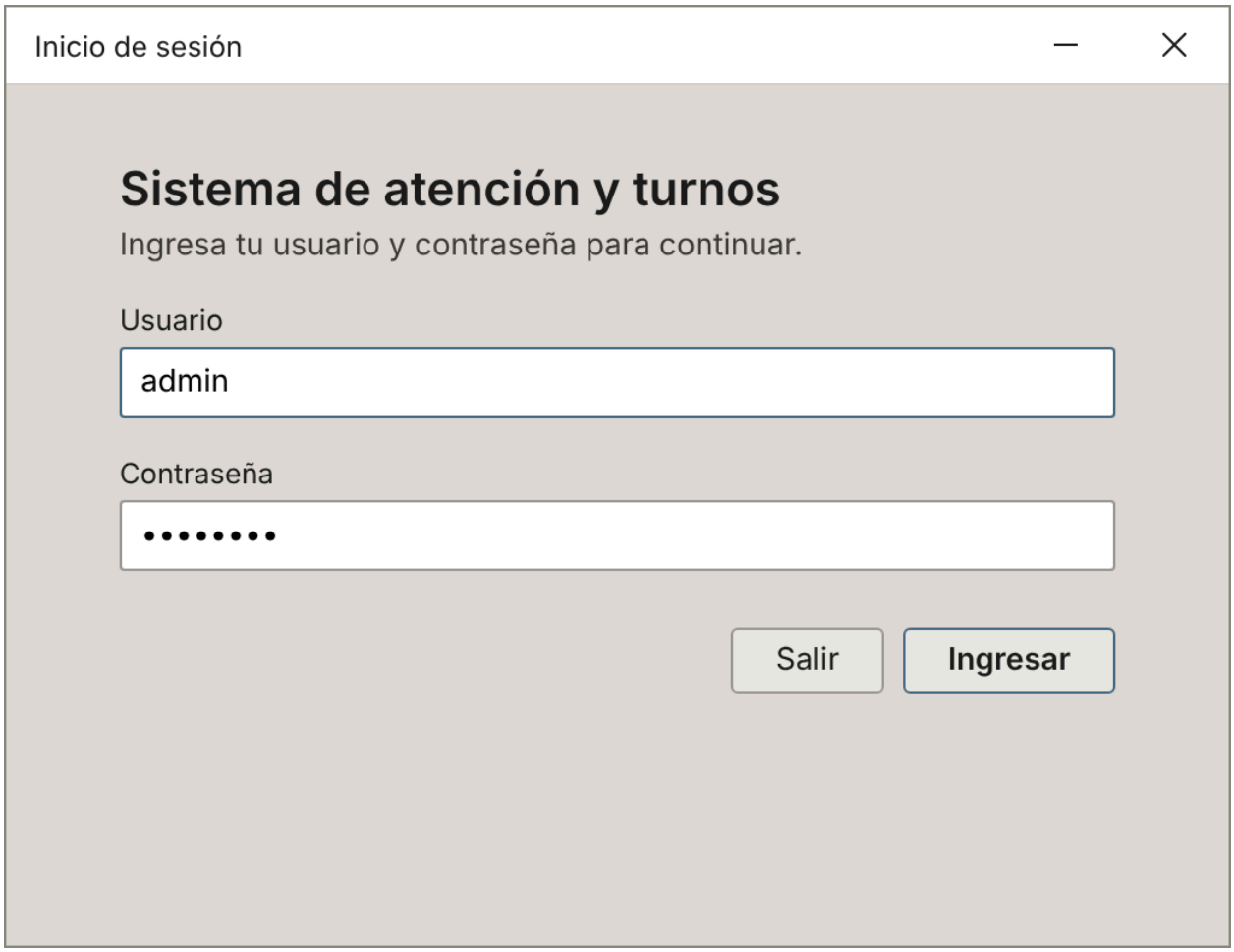
RF	Qué pide	Casos
RF-29	Historial desde la más reciente	CP-53
RF-30	Deshacer	CP-20, CP-21, CP-54, CP-56, CP-57, CP-58
RF-31	Avisar si no hay qué deshacer	CP-22
RF-32	Buscar por cinco criterios	CP-23, CP-24, CP-59 a CP-62
RF-33	Nombre parcial, sin mayúsculas ni tildes	CP-23, CP-63
RF-34	Mensaje sin resultados	CP-24
RF-35	Registro de cada atención	CP-16, CP-65
RF-36	Atendidos del más reciente al más antiguo	CP-65
RF-37	Totales de las estadísticas	CP-25, CP-66
RF-38	Colas en las estadísticas	CP-25
RF-39	Estadísticas al día	CP-25, CP-66
RF-40	Guardar después de cada cambio	CP-67
RF-41	Cargar y reconstruir colas	CP-26, CP-68
RF-42	Crear archivos y usuarios iniciales	CP-28
RF-43	Rechazar datos con formato inválido	CP-07, CP-08, CP-09, CP-34, CP-41 a CP-44, CP-71, CP-72
RF-44	Detectar números de turno repetidos al cargar	CP-69

Tres comprobaciones no se pueden hacer desde la ventana y quedan en las pruebas automáticas de la lógica. La primera es CP-18, porque un turno Atendido no aparece en las tablas de colas. La segunda es la parte de CP-19 que llama a cancelar sin pasar por el botón. La tercera es la parte de RF-31 sobre un historial sin ninguna acción: en la aplicación nunca ocurre, porque entrar ya registra un inicio de sesión.

Bocetos de la interfaz

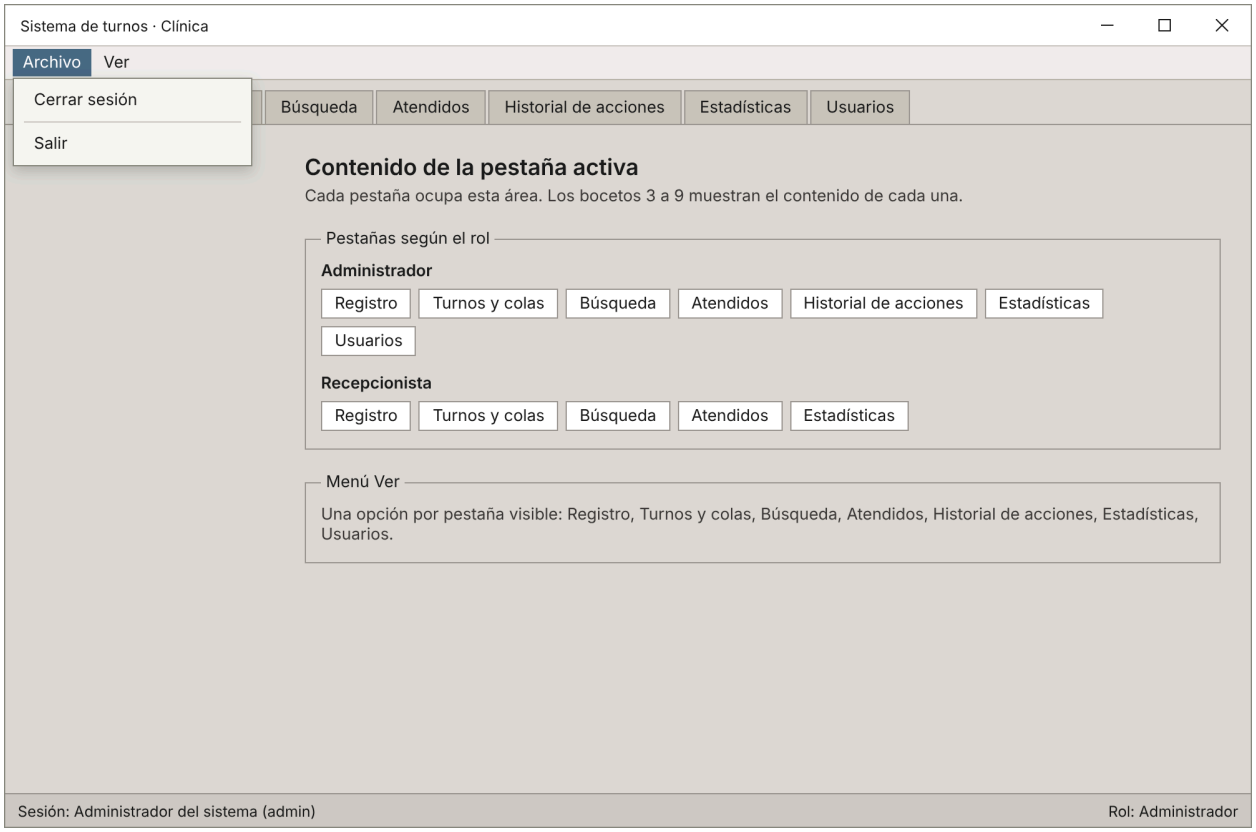
Cada pantalla imita el tema clam de ttk que usará la aplicación, así que el boceto ya se parece a lo que se va a programar. Todas las pantallas usan los mismos datos de ejemplo, que están al final de esta sección, de modo que los números de una coinciden con los de las otras.

Inicio de sesión



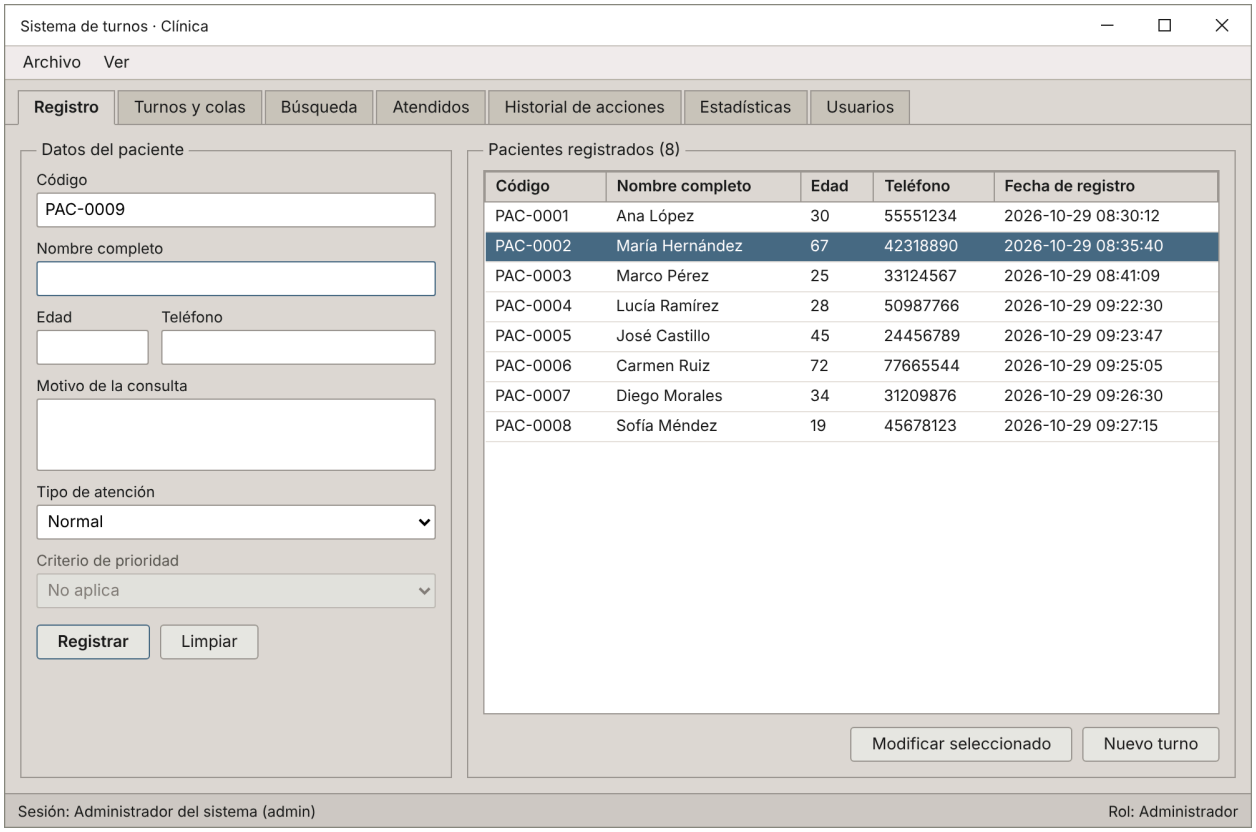
Ventana chica y centrada con el campo Usuario, el campo Contraseña y los botones Ingresar y Salir. La contraseña se escribe oculta e Ingresar también responde a Enter. Si las credenciales no coinciden, sale el mensaje "Usuario o contraseña incorrectos.", se borra la contraseña y el cursor vuelve a ese campo. Salir pide confirmación.

Ventana principal



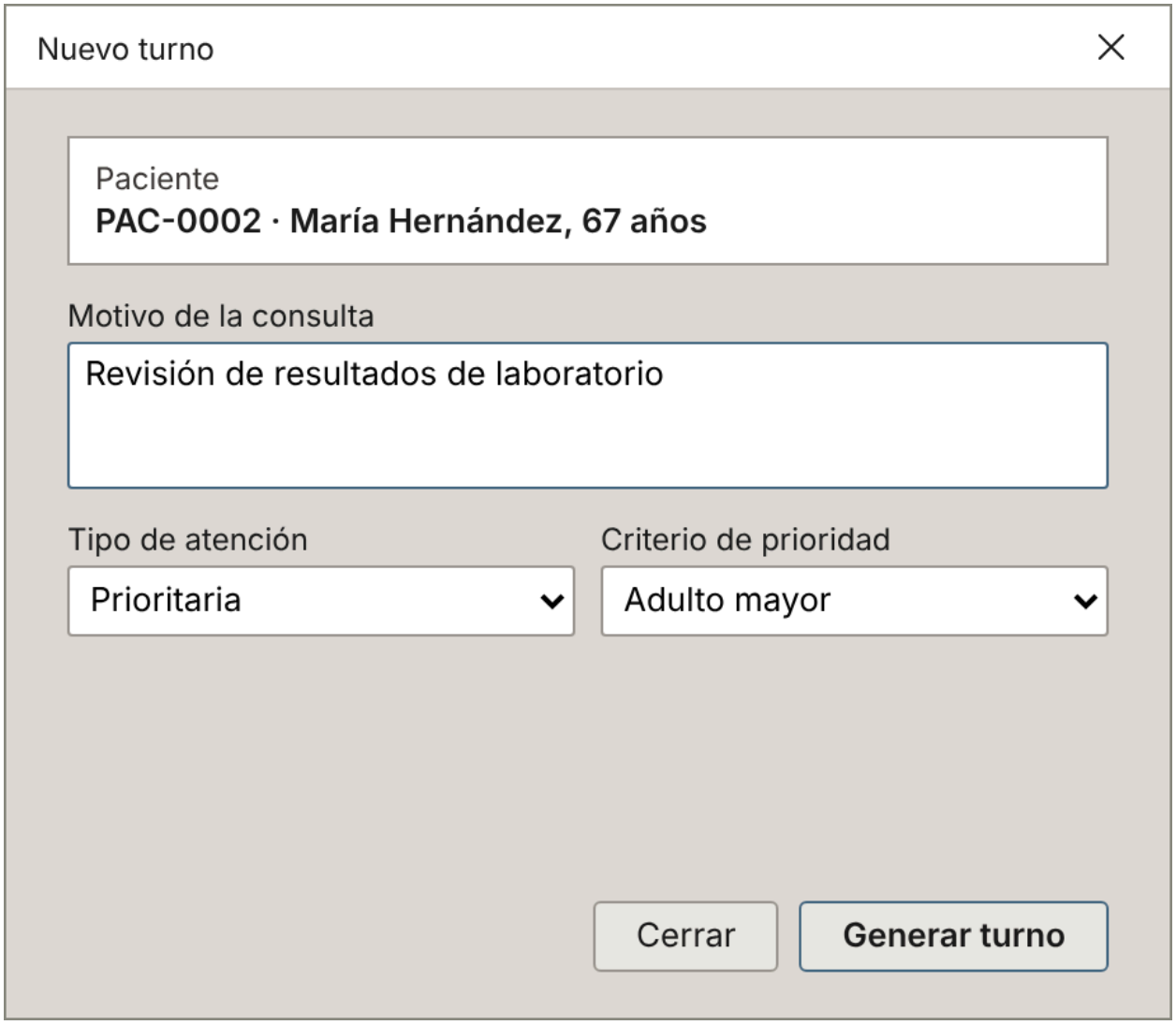
El menú Archivo trae Cerrar sesión y Salir, y el menú Ver tiene una opción por cada pestaña visible. Debajo están las pestañas. El administrador ve siete: Registro, Turnos y colas, Búsqueda, Atendidos, Historial de acciones, Estadísticas y Usuarios. El recepcionista ve cinco, porque Historial de acciones y Usuarios no se crean para su rol. La barra de estado muestra el nombre, el usuario y el rol de quien inició sesión.

Registro de pacientes



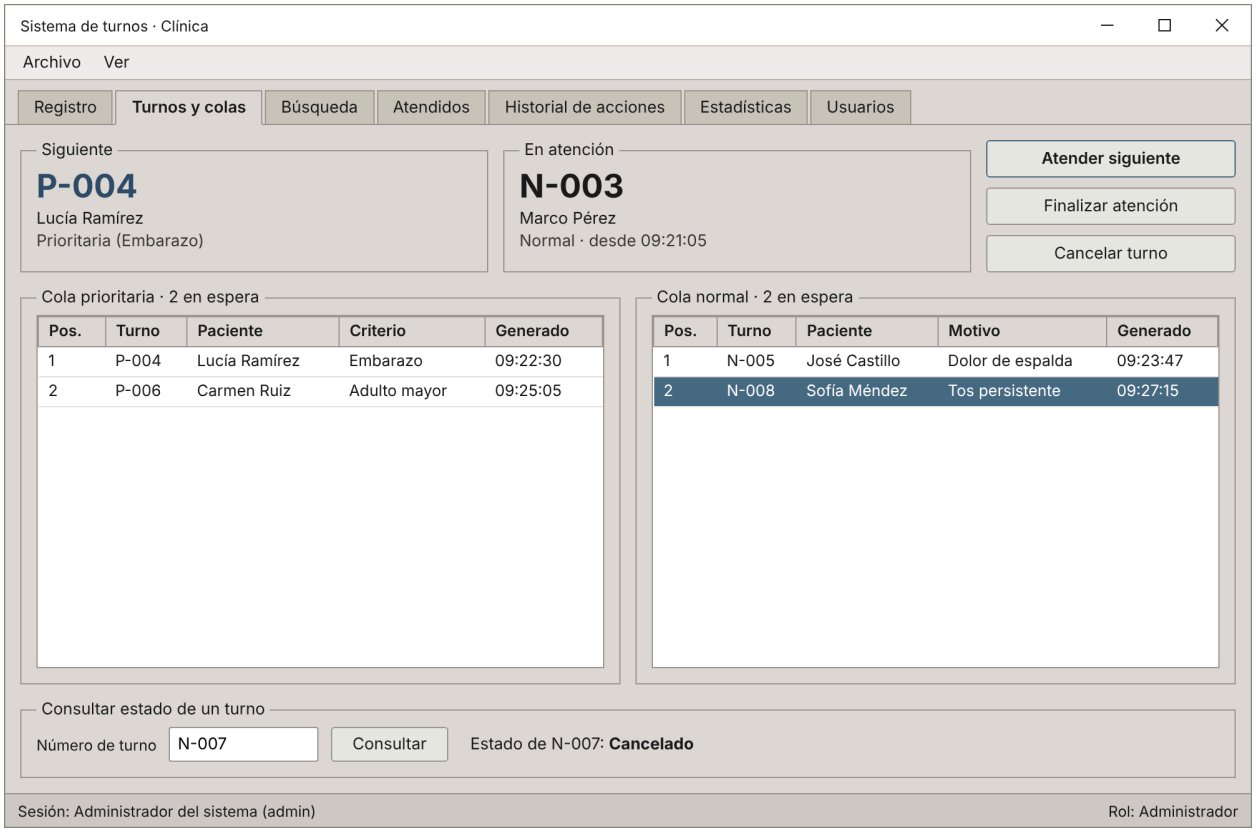
A la izquierda está el formulario: código, nombre completo, edad, teléfono, motivo de la consulta, tipo de atención y criterio de prioridad. El código llega propuesto (PAC-0009) y se puede cambiar. Criterio de prioridad queda deshabilitado mientras el tipo sea Normal. Los botones son Registrar y Limpiar. A la derecha está la tabla de pacientes con los botones Modificar seleccionado y Nuevo turno. Modificar seleccionado pasa los datos de la fila al formulario y Registrar cambia a Guardar cambios; en ese modo el código ya no se edita.

Diálogo Nuevo turno



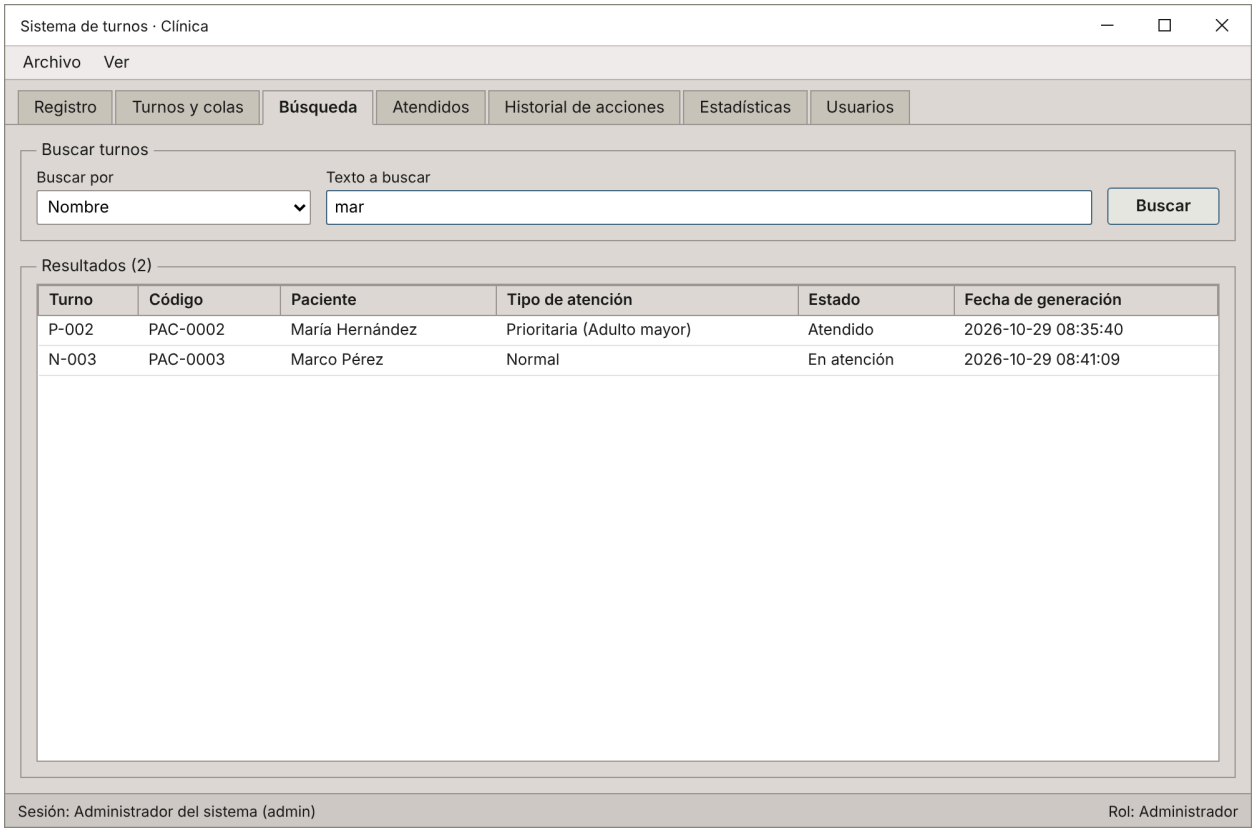
Se abre desde Registro con un paciente seleccionado. Muestra el paciente y pide motivo, tipo de atención y criterio, porque esos datos cambian en cada visita. Los botones son Generar turno y Cerrar. Si el paciente ya tiene un turno Pendiente o En atención, el sistema lo rechaza con un mensaje.

Turnos y colas



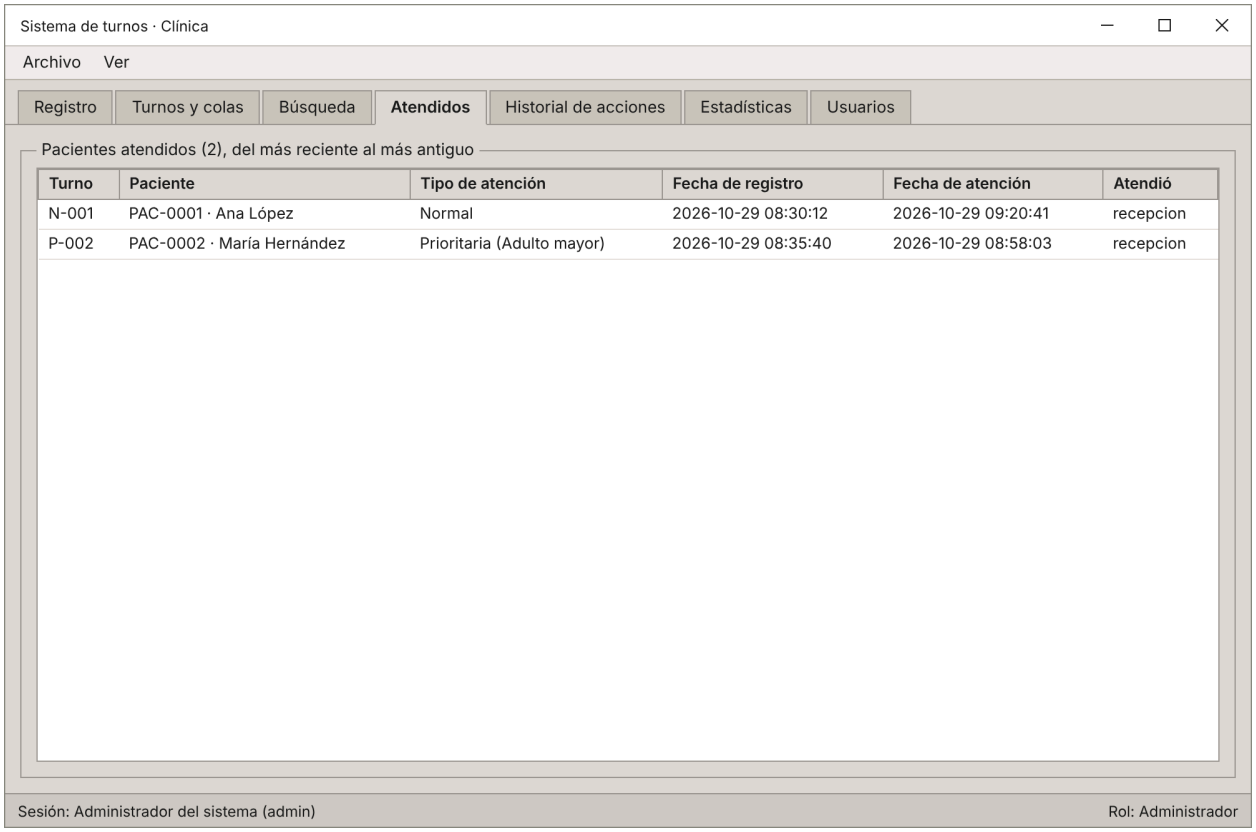
Arriba hay dos recuadros. Siguiente muestra el frente de la cola prioritaria, o el de la normal si la prioritaria está vacía, y En atención muestra el turno que está en consulta. Al lado están los botones Atender siguiente, Finalizar atención y Cancelar turno. Atender siguiente no avanza mientras haya alguien En atención. Cancelar turno actúa sobre la fila seleccionada, pide confirmación y aparece deshabilitado para el recepcionista. En el centro están las dos colas, cada una con su contador y sus turnos en orden de llegada. Abajo se consulta el estado de un turno escribiendo su número. Si las dos colas quedan vacías, el recuadro Siguiente dice "No hay pacientes en espera."

Búsqueda



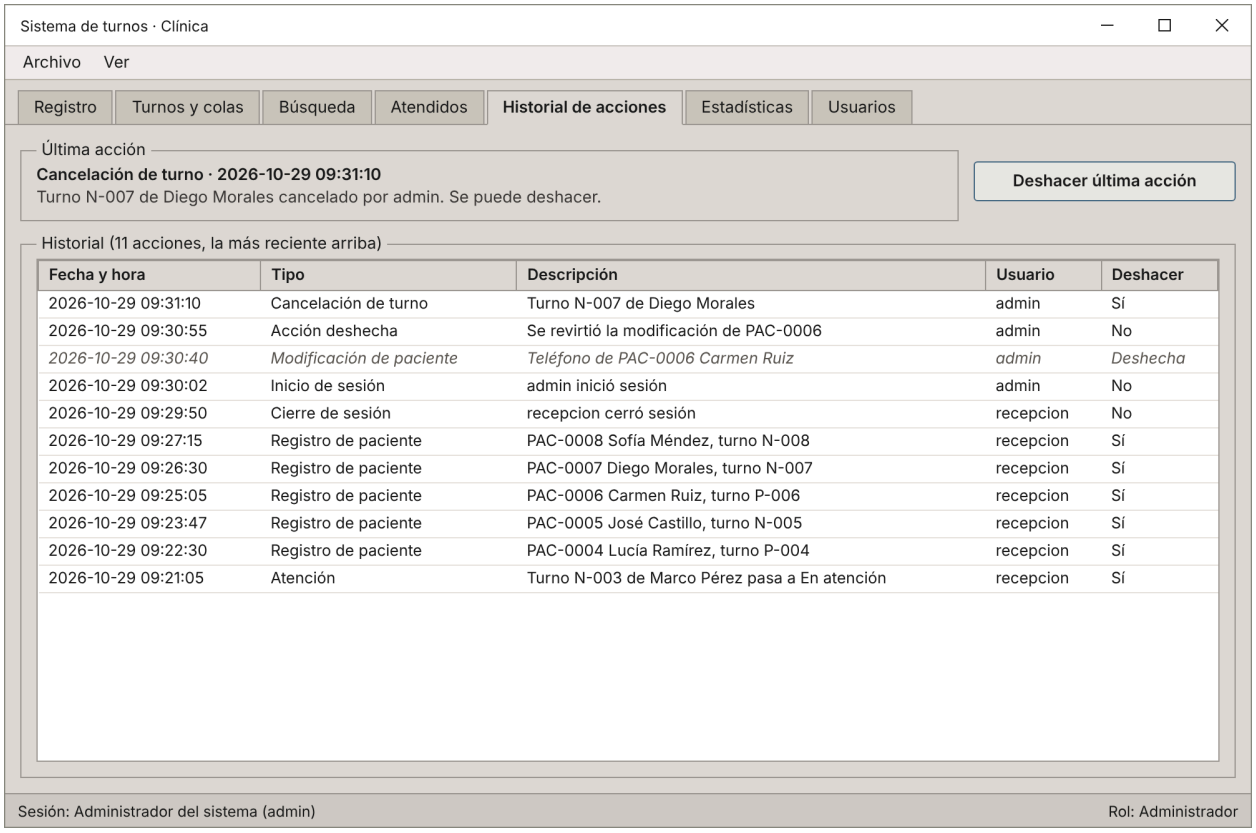
Una lista para elegir el criterio (nombre, código de paciente, número de turno, estado o tipo de atención), un campo de texto, el botón Buscar y la tabla de resultados con turno, código, paciente, tipo, estado y fecha. El nombre acepta coincidencias parciales sin importar mayúsculas ni tildes, por eso "mar" encuentra a María y a Marco. Enter también busca. Si no hay resultados, aparece un mensaje.

Historial de atendidos



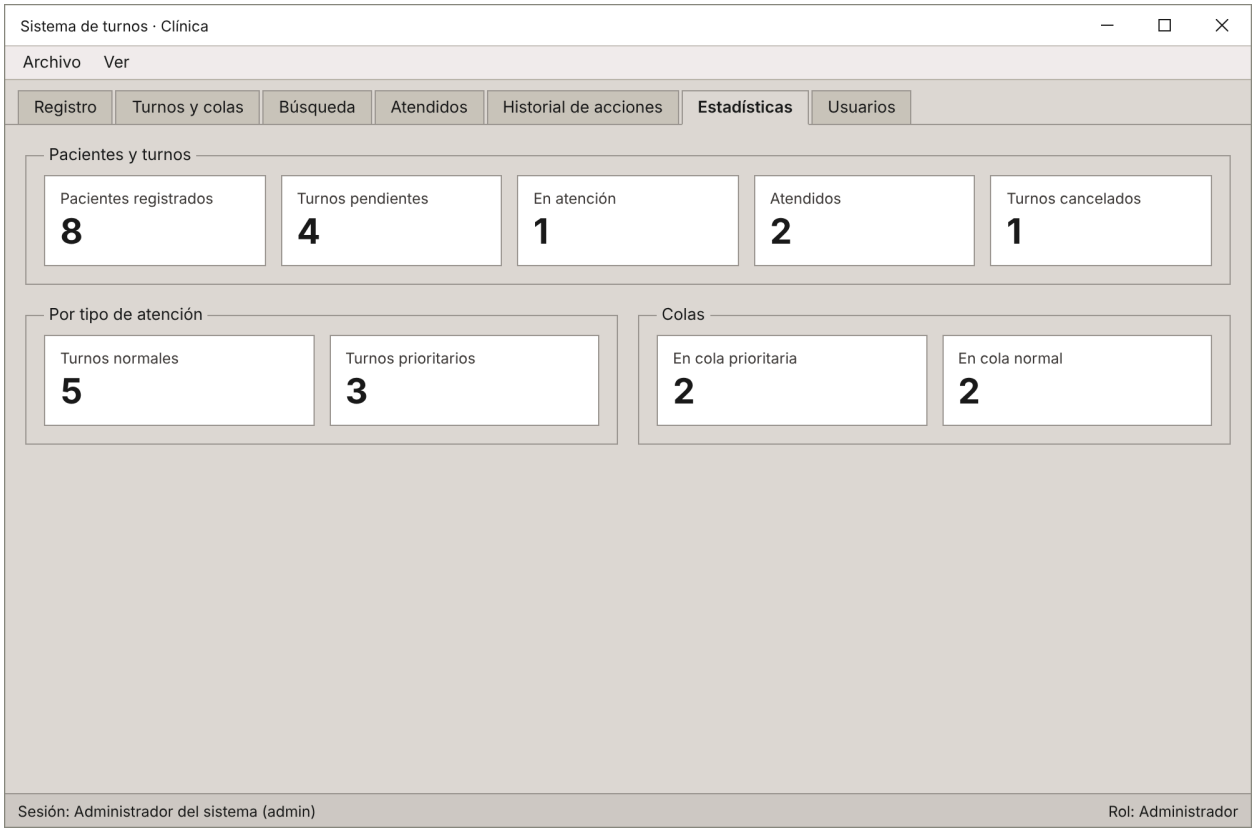
Una tabla con turno, paciente, tipo de atención, fecha de registro, fecha de atención y usuario que atendió. Hay una fila por cada atención finalizada, con la más reciente arriba. Es una pantalla de consulta y no tiene botones de edición.

Historial de acciones



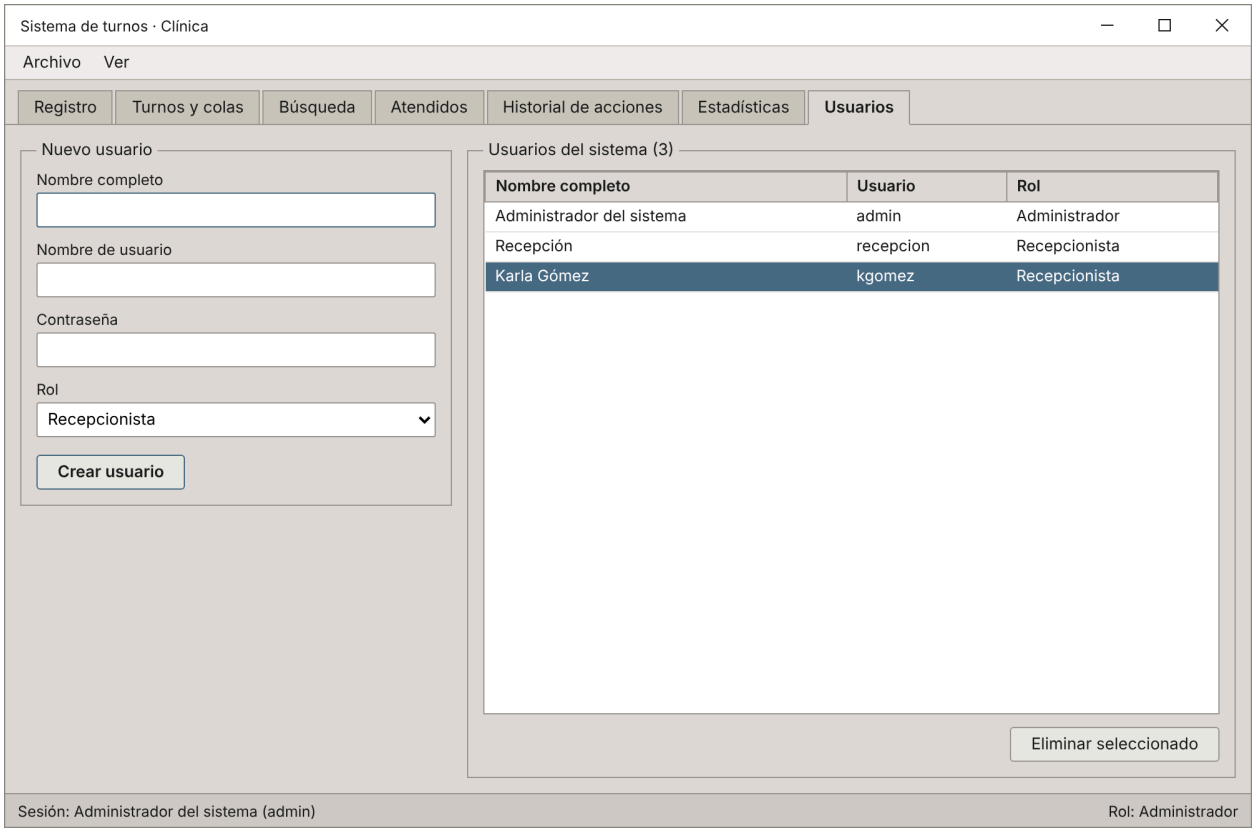
Solo la ve el administrador. Arriba está la última acción y el botón Deshacer última acción. Abajo está el historial completo con fecha, tipo, descripción, usuario y una columna Deshacer que dice Sí, No o Deshecha. Deshacer pide confirmación y revierte la última acción deshacible, aunque después se hayan registrado inicios o cierres de sesión.

Estadísticas



Etiquetas con valores, agrupadas en tres recuadros: pacientes y turnos (registrados, pendientes, en atención, atendidos y cancelados), tipo de atención (normales y prioritarios) y colas (prioritaria y normal). Se recalculan cada vez que otra pestaña cambia datos. Normales y prioritarios cuentan los turnos de ese tipo en cualquier estado.

Usuarios



Solo la ve el administrador. A la izquierda, el formulario con nombre completo, nombre de usuario, contraseña oculta y rol, y el botón Crear usuario. A la derecha, la tabla de usuarios y el botón Eliminar seleccionado, que pide confirmación y no deja borrar la cuenta propia ni quedarse sin administradores.

Datos de ejemplo de los bocetos

Paciente	Edad	Teléfono	Turno	Tipo	Estado
PAC-0001 Ana López	30	55551234	N-001	Normal	Atendido
PAC-0002 María Hernández	67	42318890	P-002	Prioritaria (Adulto mayor)	Atendido
PAC-0003 Marco Pérez	25	33124567	N-003	Normal	En atención
PAC-0004 Lucía Ramírez	28	50987766	P-004	Prioritaria (Embarazo)	Pendiente

Paciente	Edad	Teléfono	Turno	Tipo	Estado
PAC-0005 José Castillo	45	24456789	N-005	Normal	Pendiente
PAC-0006 Carmen Ruiz	72	77665544	P-006	Prioritaria (Adulto mayor)	Pendiente
PAC-0007 Diego Morales	34	31209876	N-007	Normal	Cancelado
PAC-0008 Sofía Méndez	19	45678123	N-008	Normal	Pendiente

Con estos datos las estadísticas quedan así: 8 pacientes, 4 pendientes, 1 en atención, 2 atendidos, 1 cancelado, 5 normales, 3 prioritarios, 2 en la cola prioritaria y 2 en la normal. El orden de atención respeta RN-01 y RN-02: P-002 se atendió antes que N-001 porque llegó antes de la primera llamada, y P-004 llegó después de que N-003 pasó a consulta.